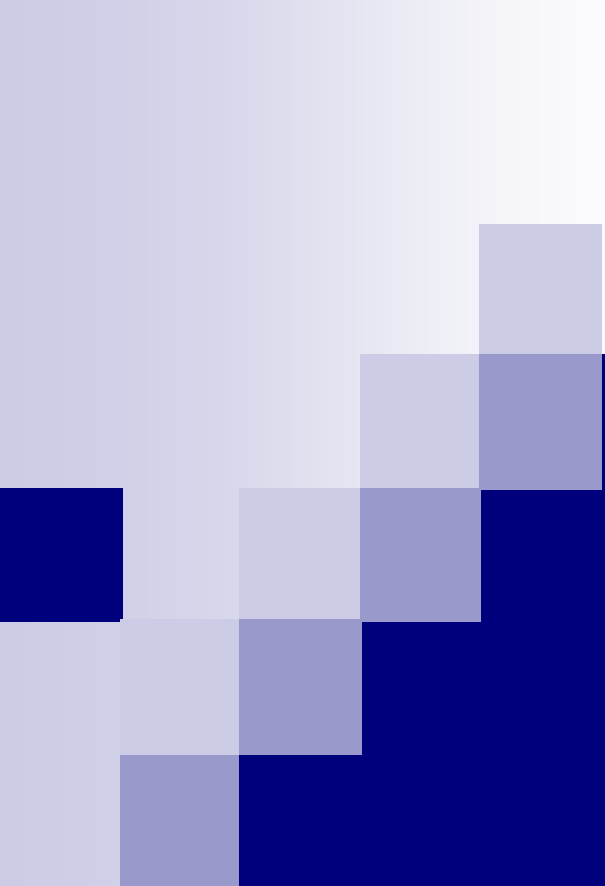
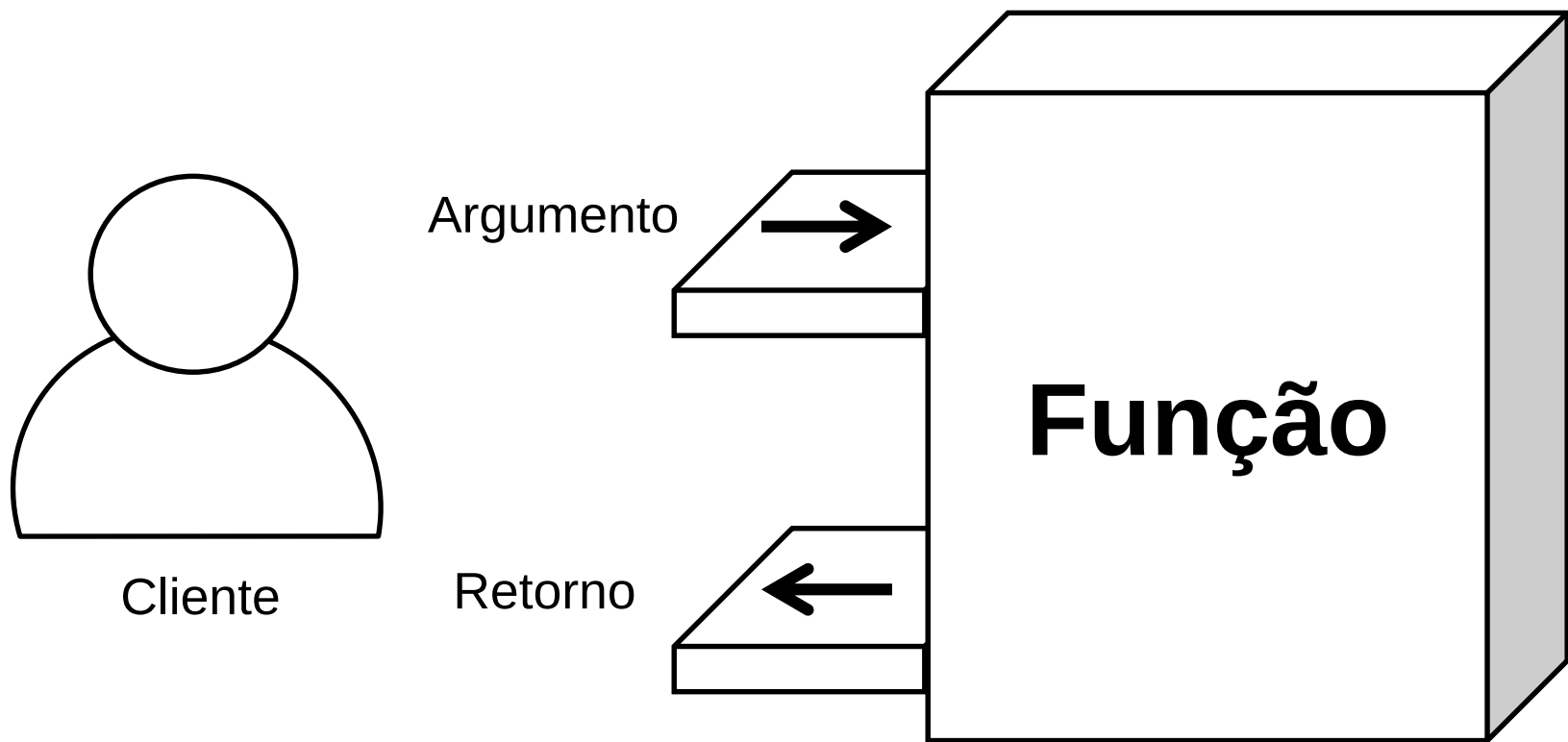




Computação Eletrônica

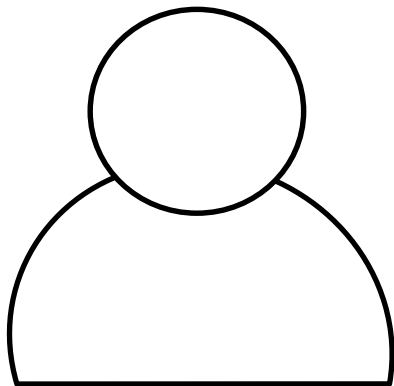
Funções

Função

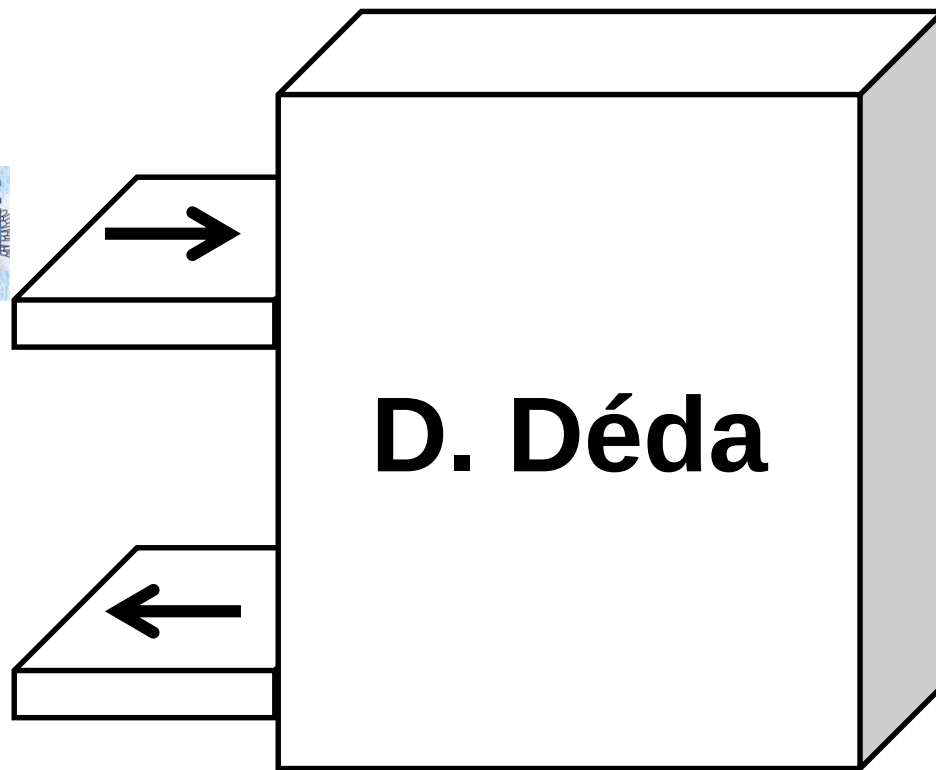


*A função é uma prestadora de serviço para o cliente.
O cliente entrega uma coisa (argumento) e ela devolve outra (retorno).*

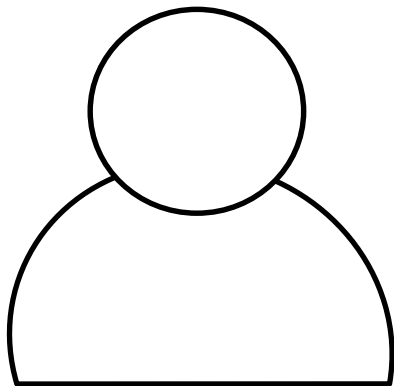
Função



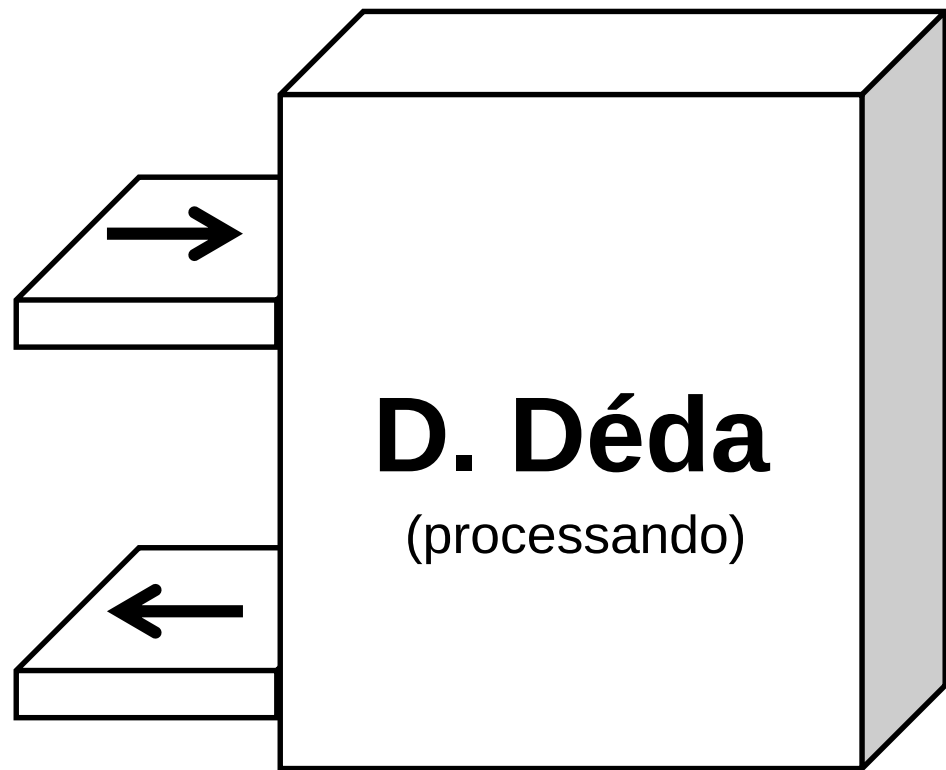
Cliente



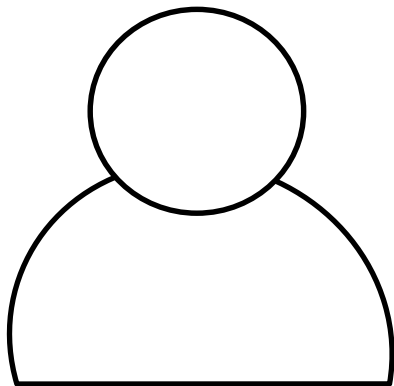
Função



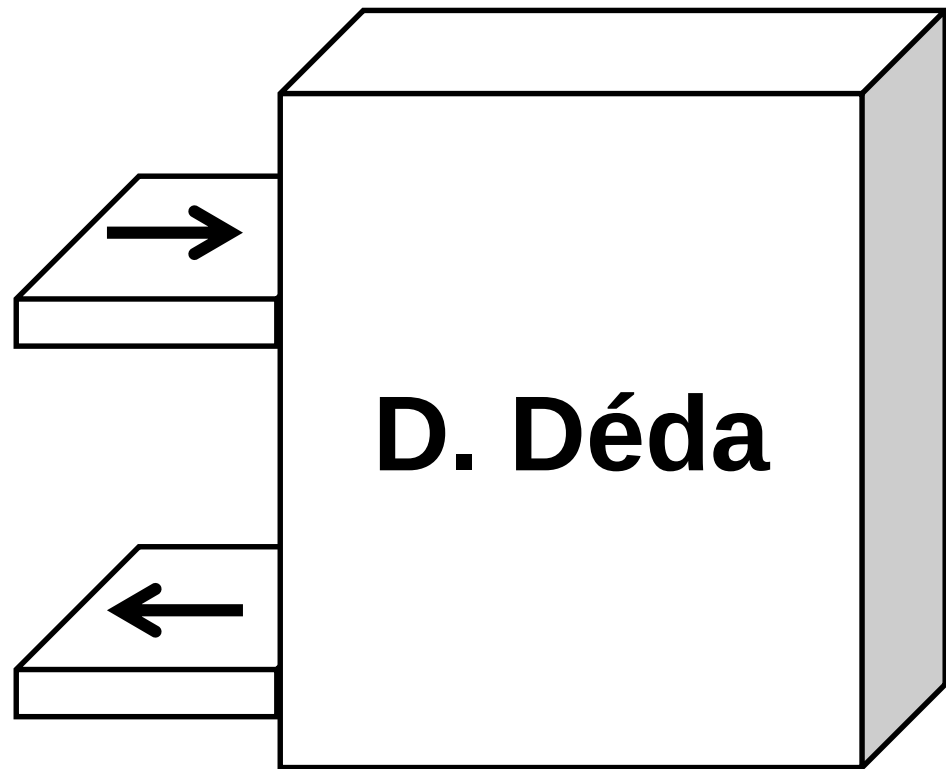
Cliente



Função

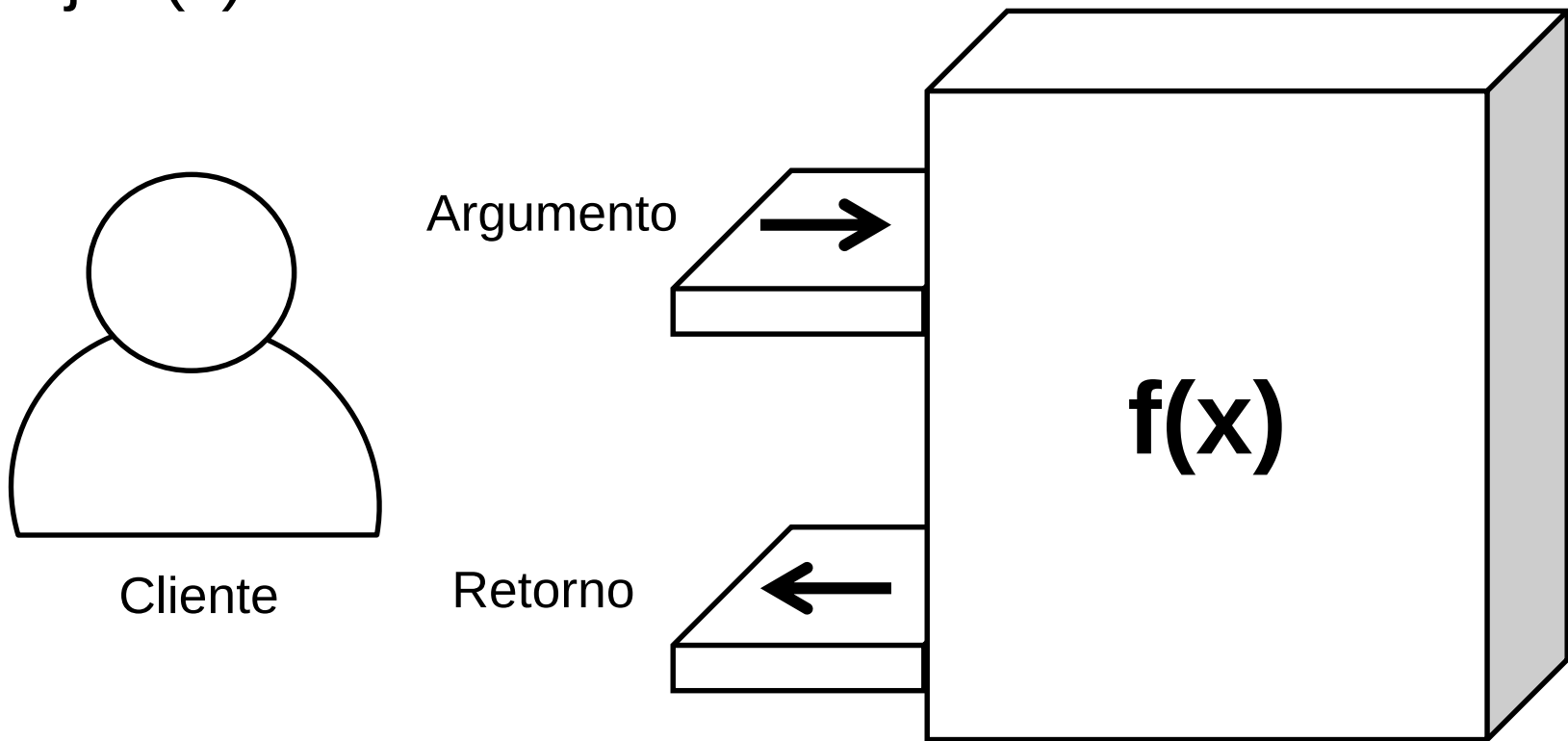


Cliente



Função

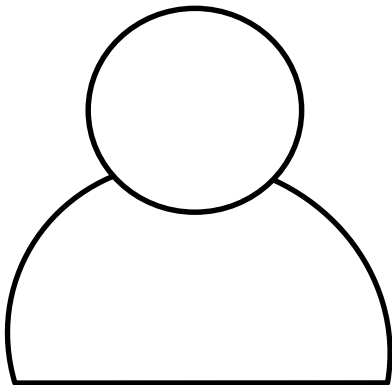
Seja $f(x) = 2x$.



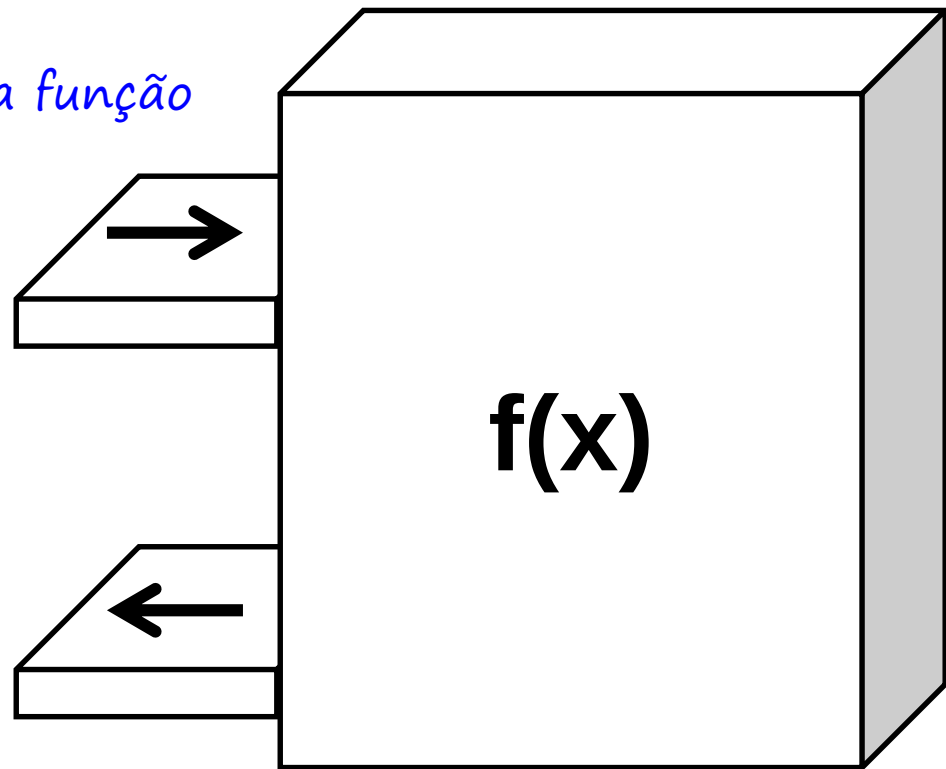
Função

Seja $f(x) = 2x$.

x é o parâmetro da função

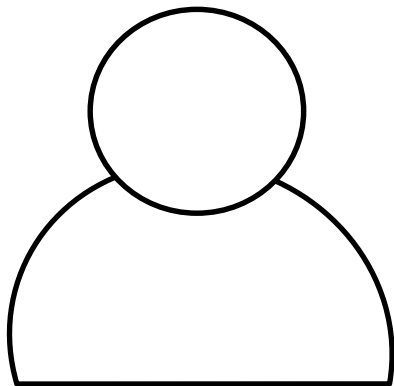


Cliente

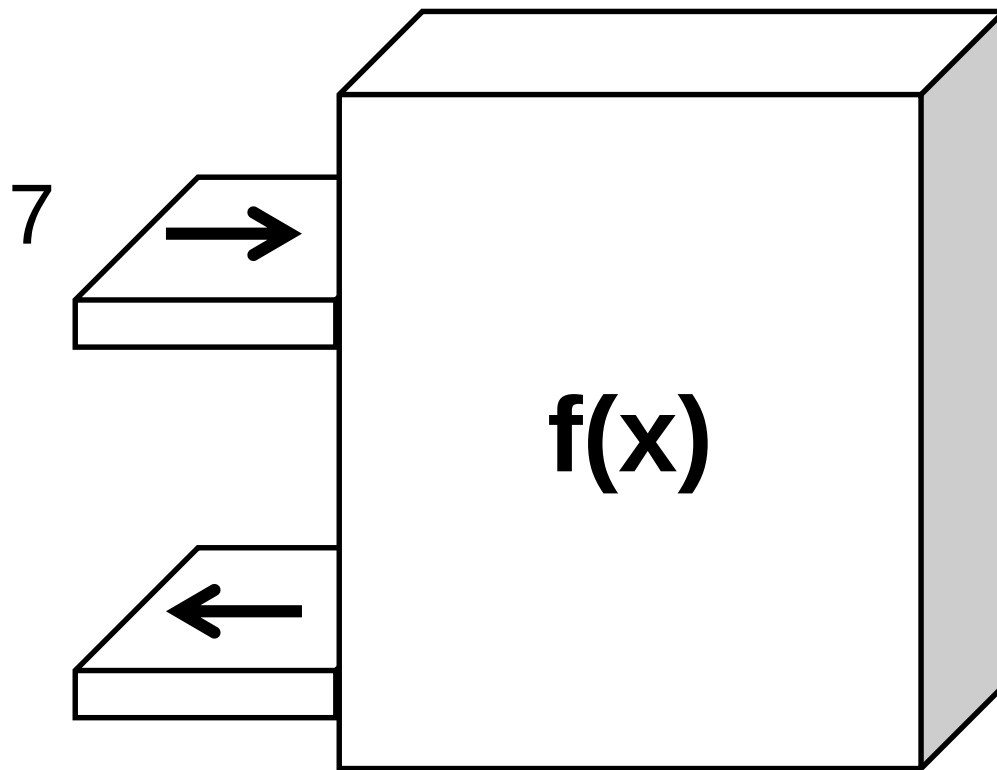


Função

Seja $f(x) = 2x$.

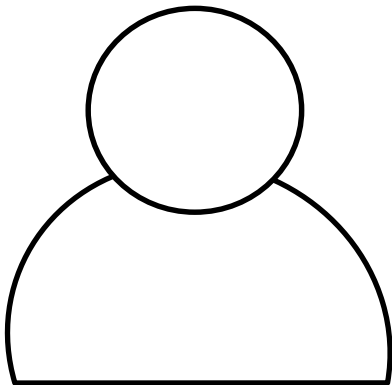


Cliente

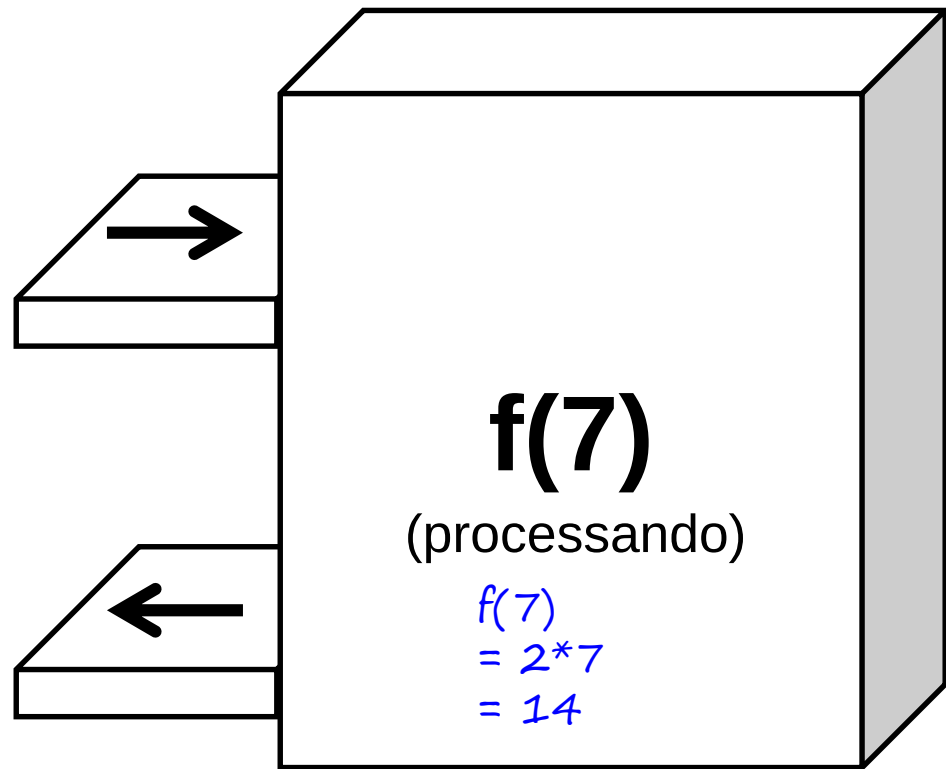


Função

Seja $f(x) = 2x$.

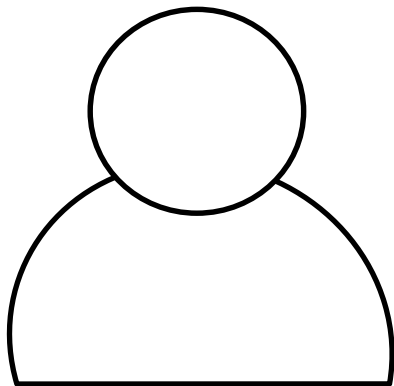


Cliente

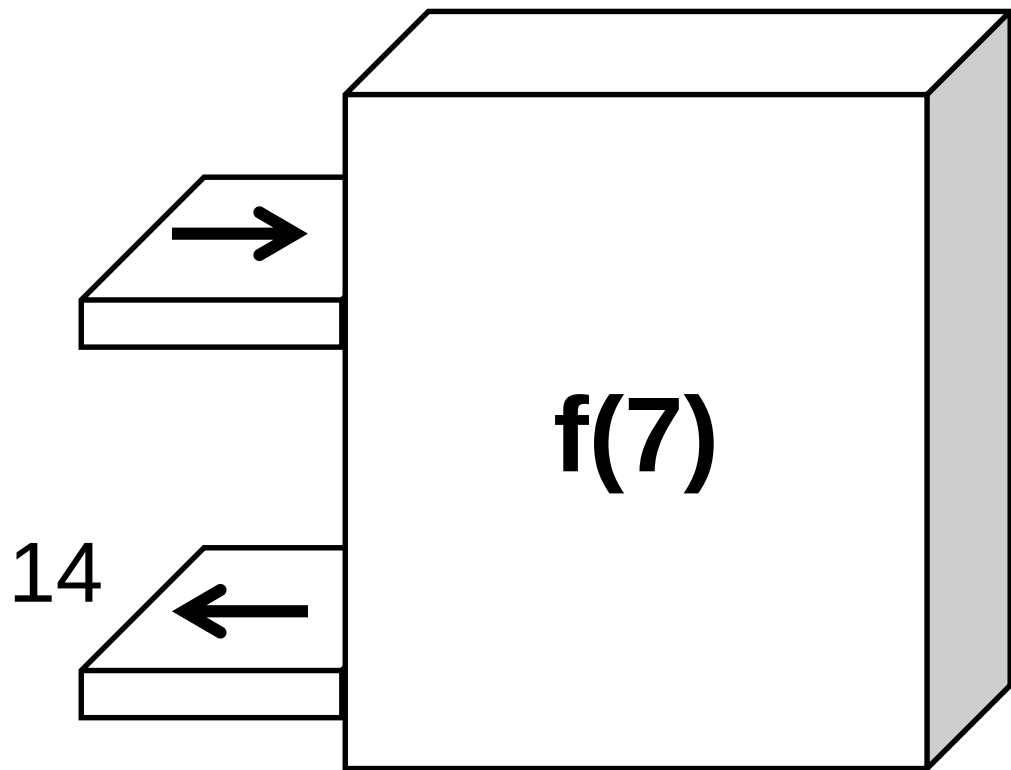


Função

Seja $f(x) = 2x$.



Cliente

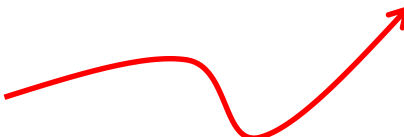


Função

```
#include <stdio.h>
```

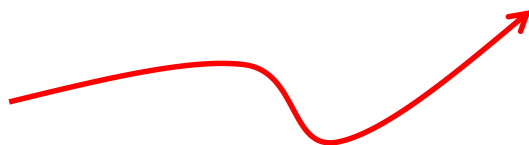
```
int f(int x) {  
    x = 2 * x;  
    return x;  
}
```

Aqui, define-se o que a função faz.



```
int main() {  
    int x, dobro;  
    printf("Entre x: ");  
    scanf("%d", &x);  
    dobro = f(x);  
    printf("%d", dobro);  
    return 0;  
}
```

O main() é o cliente.



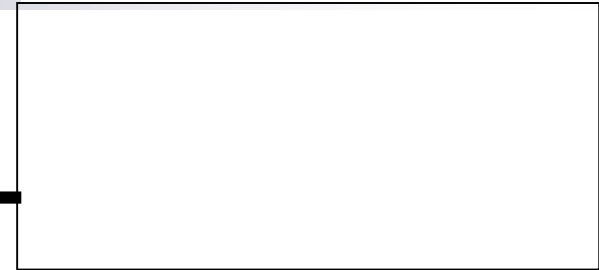
Execução

```
#include <stdio.h>
```

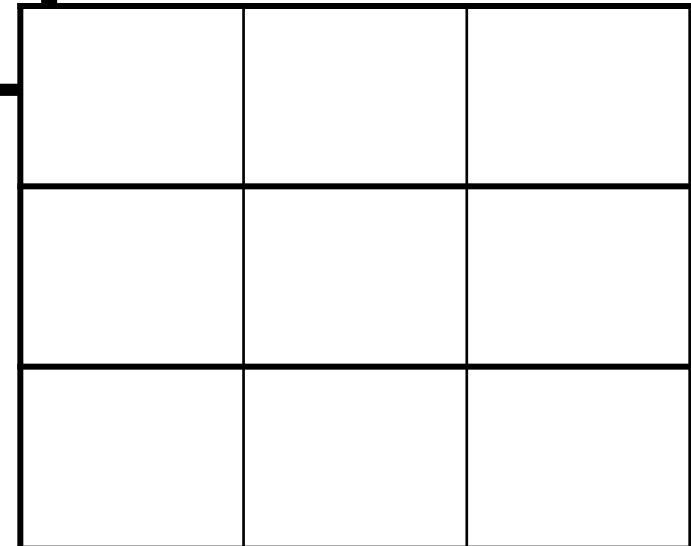
```
int f(int x) {  
    x = 2 * x;  
    return x;  
}
```



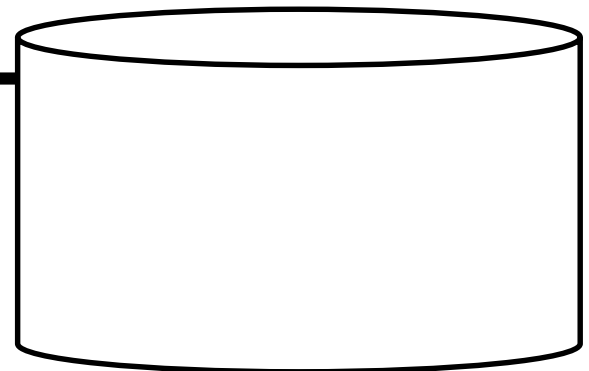
```
int main() {  
    int x, dobro;  
    printf("Entre x: ");  
    scanf("%d", &x);  
    dobro = f(x);  
    printf("%d", dobro);  
    return 0;  
}
```



Memória



HD

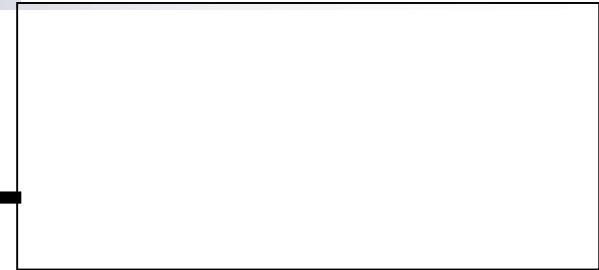


Execução

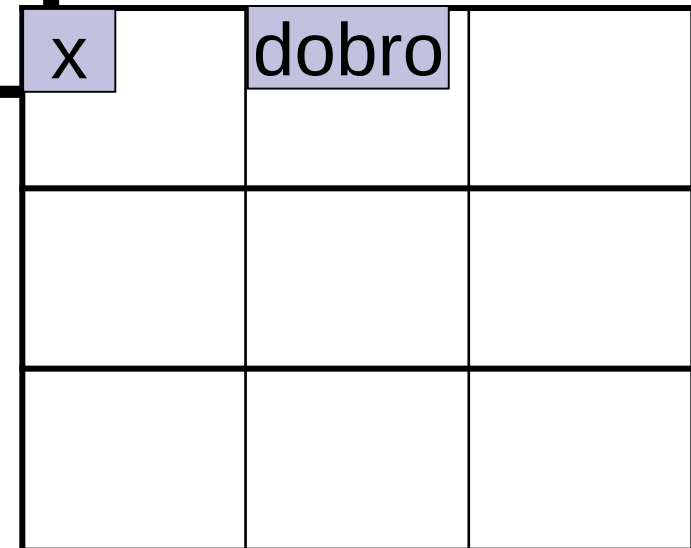
```
#include <stdio.h>
```

```
int f(int x) {  
    x = 2 * x;  
    return x;  
}
```

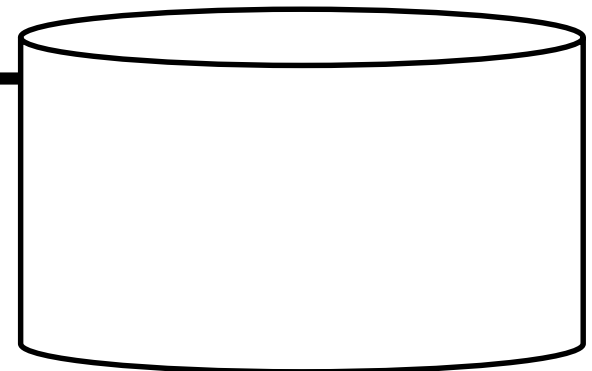
```
int main() {  
    int x, dobro;  
    printf("Entre x: ");  
    scanf("%d", &x);  
    dobro = f(x);  
    printf("%d", dobro);  
    return 0;  
}
```



Memória



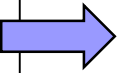
HD



Execução

```
#include <stdio.h>
```

```
int f(int x) {  
    x = 2 * x;  
    return x;  
}
```



```
int main() {  
    int x, dobro;  
    printf("Entre x: ");  
    scanf("%d", &x);  
    dobro = f(x);  
    printf("%d", dobro);  
    return 0;  
}
```

Entre x:

Memória

x

dobro

HD

Execução

```
#include <stdio.h>

int f(int x) {
    x = 2 * x;
    return x;
}

int main() {
    int x, dobro;
    printf("Entre x: ");
    scanf("%d", &x);
    dobro = f(x);
    printf("%d", dobro);
    return 0;
}
```

Entre x: 7

Memória

x

dobro

HD

Entre x: 7 <ENTER>

Execução

```
#include <stdio.h>

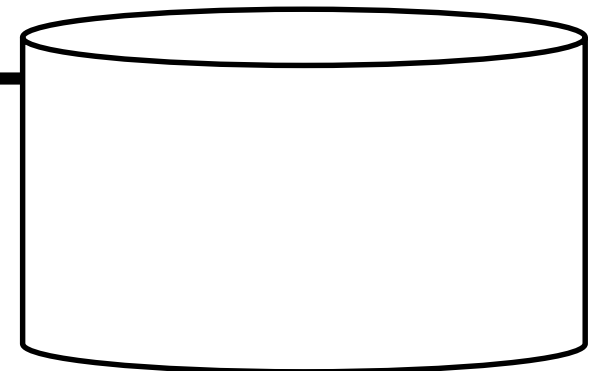
int f(int x) {
    x = 2 * x;
    return x;
}

int main() {
    int x, dobro;
    printf("Entre x: ");
    scanf("%d", &x);
    dobro = f(x);
    printf("%d", dobro);
    return 0;
}
```

Memória

x	7	dobro	

HD



Entre x: 7 <ENTER>

Execução

```
#include <stdio.h>
```

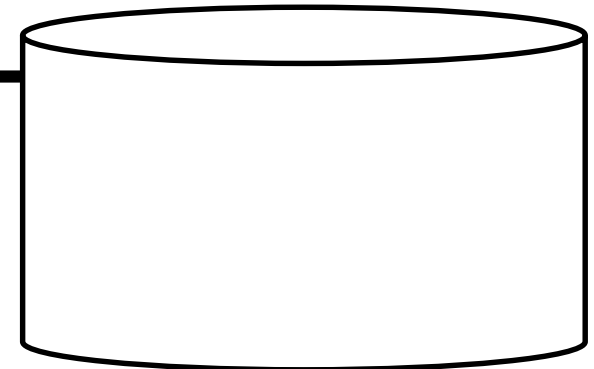
```
int f(int x) {  
    x = 2 * x;  
    return x;  
}
```

```
int main() {  
    int x, dobro;  
    printf("Entre x: ");  
    scanf("%d", &x);  
    dobro = f(x);  
    printf("%d", dobro);  
    return 0;  
}
```

Memória

x	7	dobro	

HD



O computador
chama f(7)
para trabalhar

Entre x: 7 <ENTER>

Execução

```
#include <stdio.h>
```

```
int f(int x) {  
    x = 2 * x;  
    return x;  
}
```

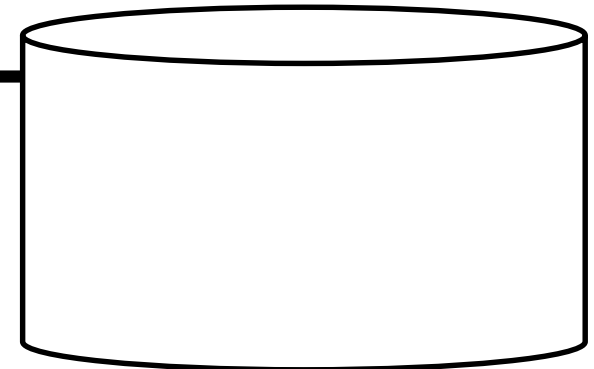
```
int main() {  
    int x, dobro;  
    printf("Entre x: ");  
    scanf("%d", &x);  
    dobro = f(x);  
    printf("%d", dobro);  
    return 0;  
}
```

O main() fica parado aqui, aguardando a função f(7) retornar

Memória

x	7	dobro	

HD



Entre x: 7 <ENTER>

Execução

```
#include <stdio.h>
```

```
int f(int x) {
    x = 2 * x;
    return x;
}
```

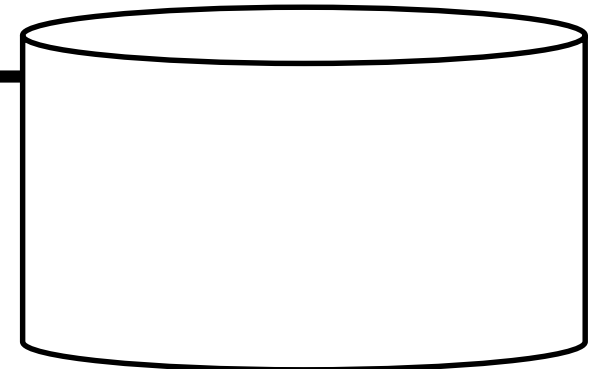
Outra variável x é criada na memória. Esta variável é exclusiva da função f() e ocupa outro espaço na memória diferente do x da função main(). O valor 7 passado como argumento para f() é armazenado neste x. Para não confundirmos, chamaremos o x da função f() de $x_{f()}$.

```
int main() {
    int x, dobro;
    printf("Entre x: ");
    scanf("%d", &x);
    dobro = f(x);
    printf("%d", dobro);
    return 0;
}
```

Memória

x	dobro	$x_{f()}$
7		7

HD



Entre x: 7 <ENTER>

Execução

```
#include <stdio.h>
```

```
int f(int x) {
```

```
    x = 2 * x;
```

```
    return x;
```

```
}
```

```
int main() {
```

```
    int x, dobro;
```

```
    printf("Entre x: ");
```

```
    scanf("%d", &x);
```

```
    dobro = f(x);
```

```
    printf("%d", dobro);
```

```
    return 0;
```

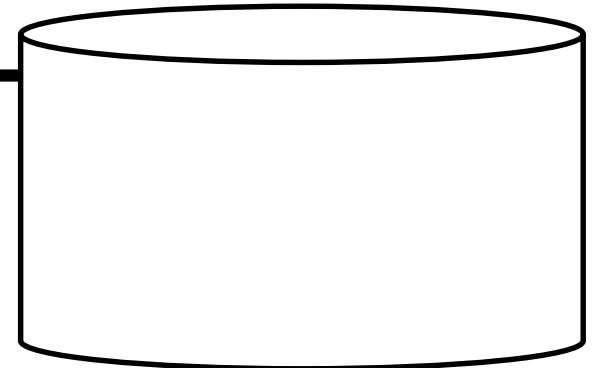
```
}
```

$2 * x_{f()}$
 $= 2 * 7$
 $= 14$

Memória

x	dobro	$x_{f()}$
7		7

HD



Entre x: 7 <ENTER>

Execução

```
#include <stdio.h>
```

```
int f(int x) {  
    x = 2 * x;  
    return x;  
}
```

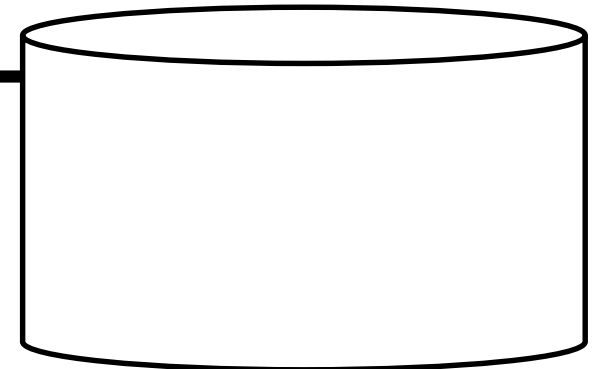
```
int main() {  
    int x, dobro;  
    printf("Entre x: ");  
    scanf("%d", &x);  
    dobro = f(x);  
    printf("%d", dobro);  
    return 0;  
}
```

$$\begin{aligned} 2 * x \\ = 2 * 7 \\ = 14 \end{aligned}$$

Memória

x	dobro	x _{f()}
7		14

HD



Entre x: 7 <ENTER>

Execução

```
#include <stdio.h>
```

```
int f(int x) {  
    x = 2 * x;  
    return x;  
}
```

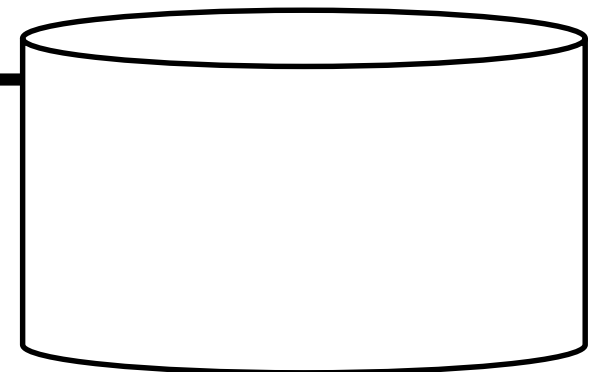
*O valor de $x_{f()}$ é
retornado ao main() no
lugar da chamada de f().*

```
int main() {  
    int x, dobro;  
    printf("Entre x: ");  
    scanf("%d", &x);  
    dobro = f(x);  
    printf("%d", dobro);  
    return 0;  
}
```

Memória

x	dobro	$x_{f()}$
7		14

HD



Entre x: 7 <ENTER>

Execução

```
#include <stdio.h>
```

```
int f(int x) {
    x = 2 * x;
    return x;
}
```

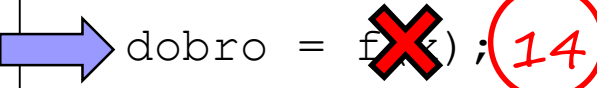
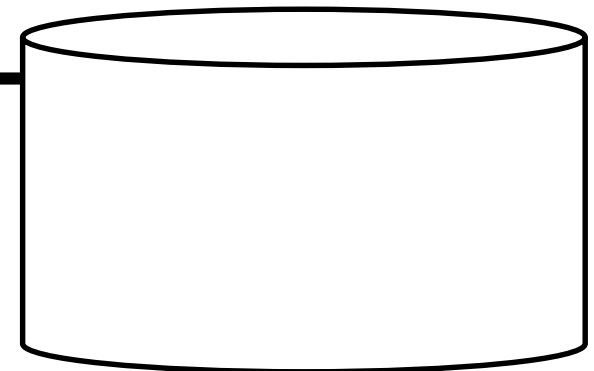
14 é atribuído à variável dobro e o $x_{f()}$ é apagado da memória.

```
int main() {
    int x, dobro;
    printf("Entre x: ");
    scanf("%d", &x);
    dobro = f(x);
    printf("%d", dobro);
    return 0;
}
```

Memória

x	7	dobro	
		14	

HD



Execução

```
#include <stdio.h>
```

```
int f(int x) {  
    x = 2 * x;  
    return x;  
}
```

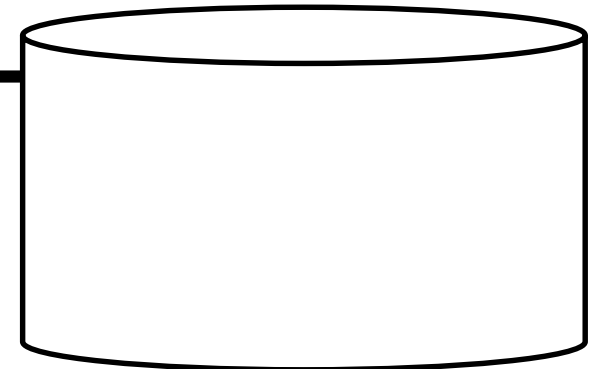
```
int main() {  
    int x, dobro;  
    printf("Entre x: ");  
    scanf("%d", &x);  
    dobro = f(x);  
    printf("%d", dobro);  
    return 0;  
}
```

Entre x: 7 <ENTER>
14

Memória

x	7	dobro	
		14	

HD



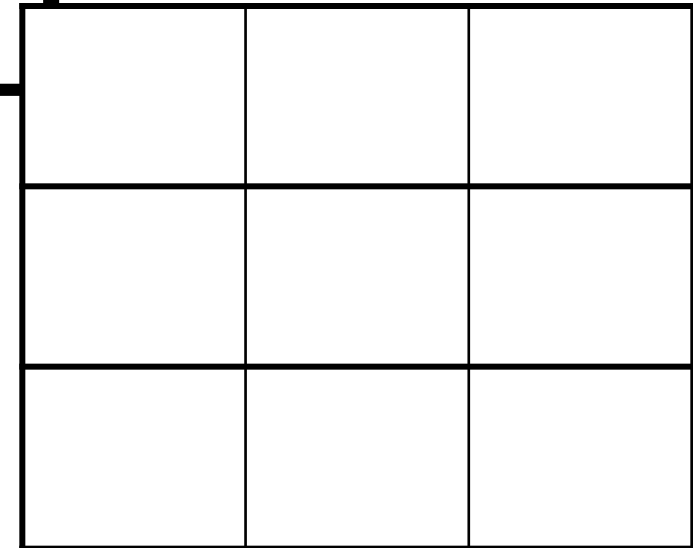
Execução

```
#include <stdio.h>

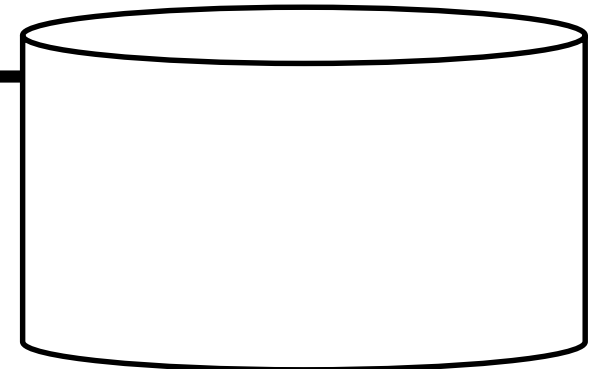
int f(int x) {
    x = 2 * x;
    return x;
}

int main() {
    int x, dobro;
    printf("Entre x: ");
    scanf("%d", &x);
    dobro = f(x);
    printf("%d", dobro);
    return 0;
}
```

Memória



HD





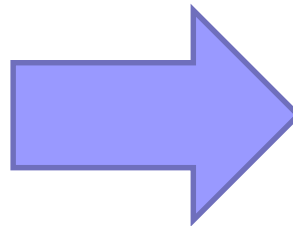
Função

- Modularização

- ☐ Módulos são responsáveis por um *pedaço* da solução
- ☐ Dividir para conquistar
- ☐ Reutilização de código
- ☐ Facilidade de leitura e correção de erros

- Em C, modularização pode ser obtida usando funções

Modularização



Função

- Uma função possui um conjunto de instruções para realizar uma ou mais tarefas e é referenciada por um único nome
- `main()`, `printf()`, `scanf()` são exemplos de funções
- Funções podem ser ativadas (chamadas) pela função `main()` ou outras funções

Função

- Uma função em C pode retornar somente um valor
 - `int`, `double`, `float`, `char`, ...
- Nem todas funções precisam retornar um valor
 - Nesta caso, utiliza-se o tipo `void`
 - `return` pode ser omitido na função
 - `printf()` e `scanf()` são exemplos de funções que não retornam nada

Função

Nome

Parâmetros

```
int f(int x) {  
    x = 2 * x;  
    return x;  
}
```

Tipo de retorno

Retorno

Função

- Pra chamar a função, deve-se utilizar o nome e os argumentos que ela precisa entre parênteses
- Os argumentos precisam ser na ordem dos parâmetros que a função tem
 - Exemplo: $f(3)$

Função

■ O que este programa faz?

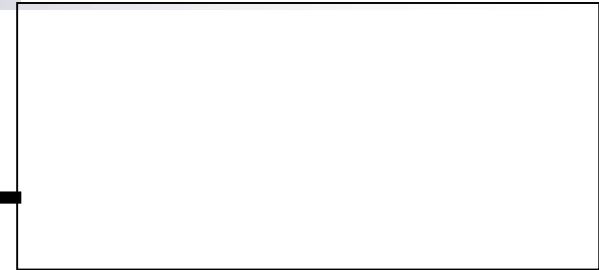
```
#include <stdio.h>

void hello() {
    printf("Hello world\n");
    return;
}

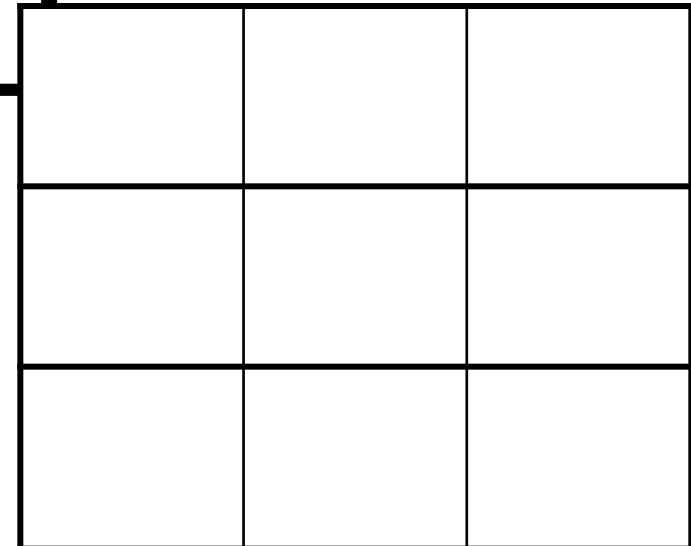
int main() {
    hello();
    hello();
    return 0;
}
```


Execução

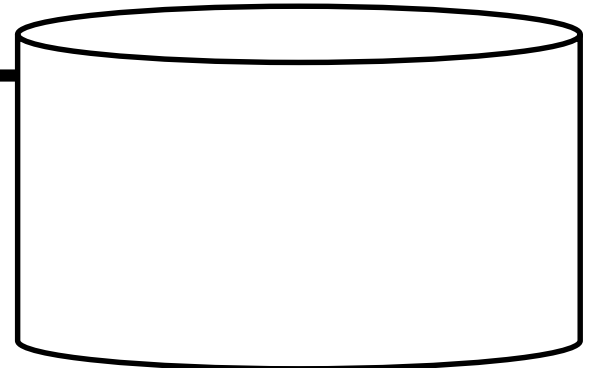
```
#include <stdio.h>
void hello() {
    printf("Hello world\n");
    return;
}
→ int main() {
    hello();
    hello();
    return 0;
}
```



Memória



HD



Execução

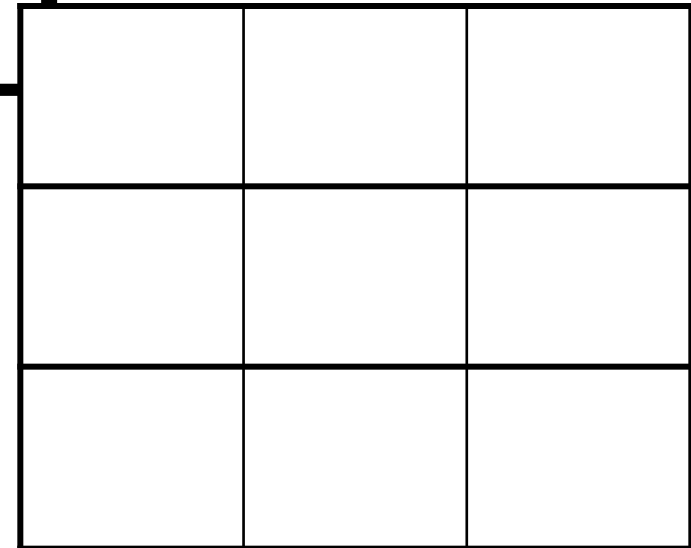
```
#include <stdio.h>

void hello() {
    printf("Hello world\n");
    return;
}
```

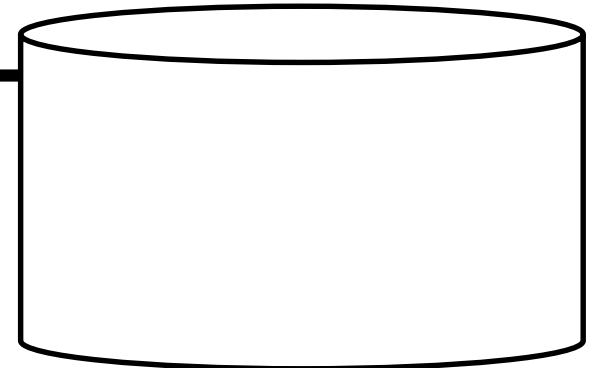
```
int main() {
    hello();
    hello();
    return 0;
}
```

*O computador
chama hello()
para trabalhar*

Memória



HD



Execução

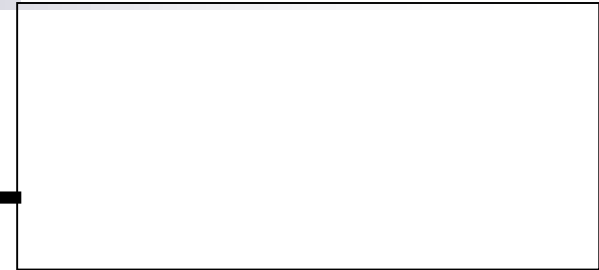
```
#include <stdio.h>
```

```
void hello() {  
    printf("Hello world\n");  
    return;  
}
```

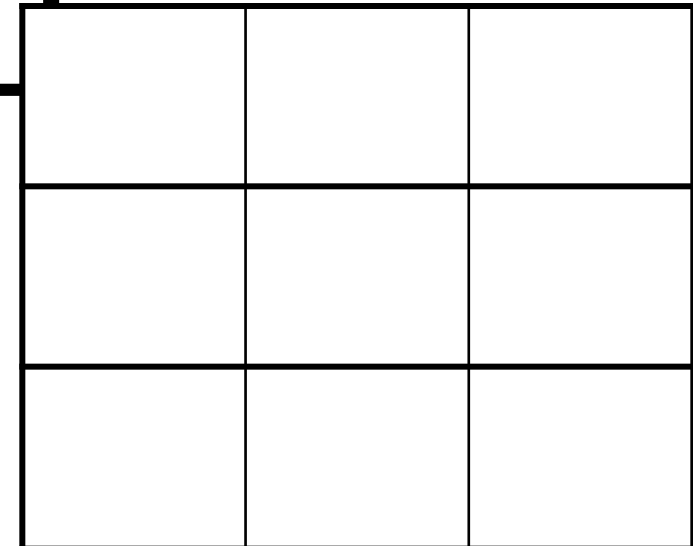
```
int main() {
```

```
    hello();  
    hello();  
    return 0;  
}
```

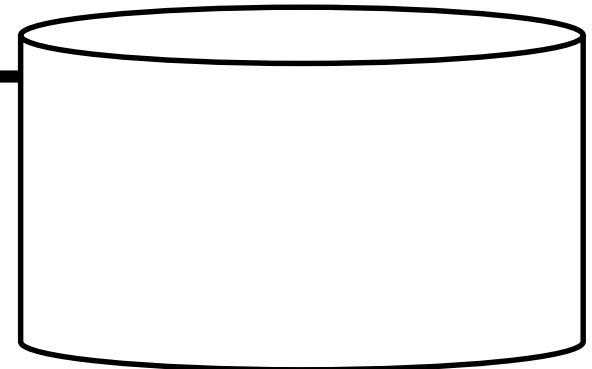
O main() fica parado aqui, aguardando a função hello() retornar



Memória



HD



Execução

```
#include <stdio.h>
```

```
void hello() {
```

```
→ printf("Hello world\n");
```

```
    return;
```

```
}
```

```
int main() {
```

```
→ hello();
```

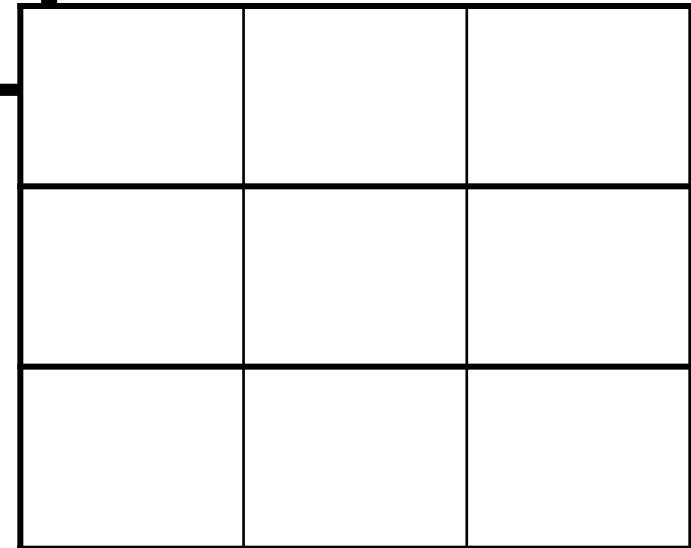
```
    hello();
```

```
    return 0;
```

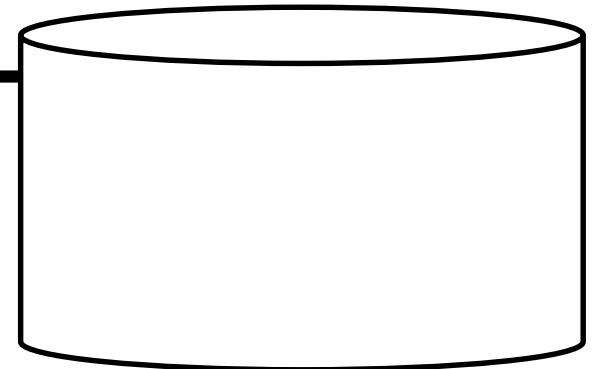
```
}
```

Hello world

Memória



HD



Execução

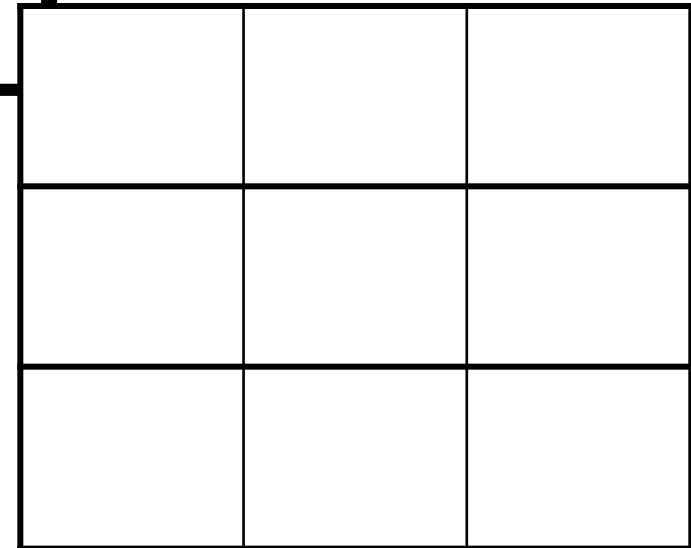
```
#include <stdio.h>

void hello() {
    printf("Hello world\n");
    → return;
}
```

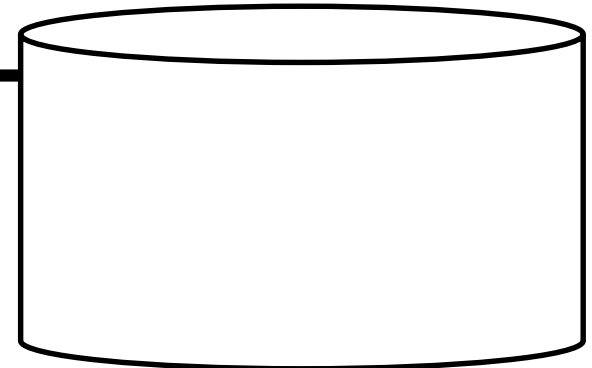
```
int main() {
    → hello();
    hello();
    return 0;
}
```

Hello world

Memória



HD



Execução

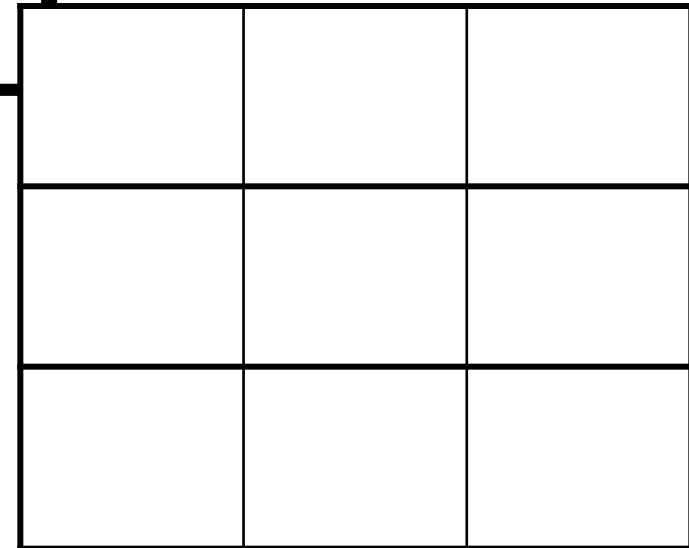
```
#include <stdio.h>

void hello() {
    printf("Hello world\n");
    return;
}

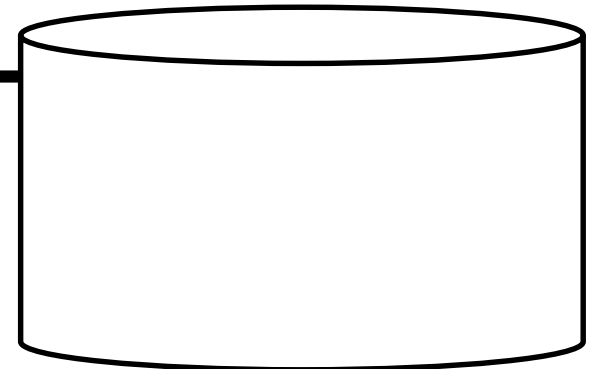
int main() {
    → hello();
    hello();
    return 0;
}
```

Hello world

Memória



HD



Execução

```
#include <stdio.h>

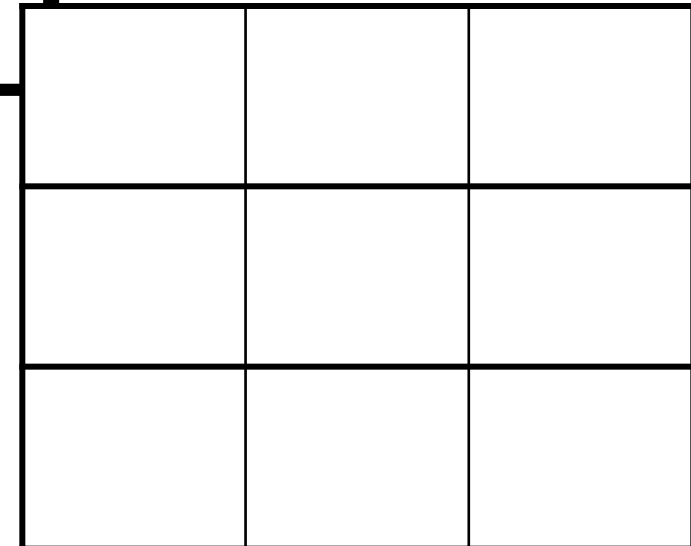
void hello() {
    printf("Hello world\n");
    return;
}
```

```
int main() {
    hello();
    hello();
    return 0;
}
```

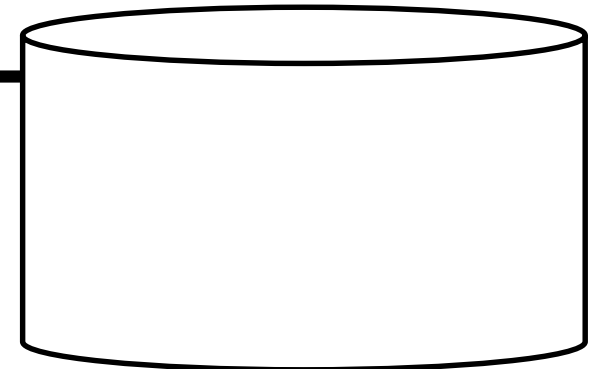
*O computador
chama hello() pela
segunda vez para
trabalhar*

Hello world

Memória



HD



Execução

```
#include <stdio.h>
```

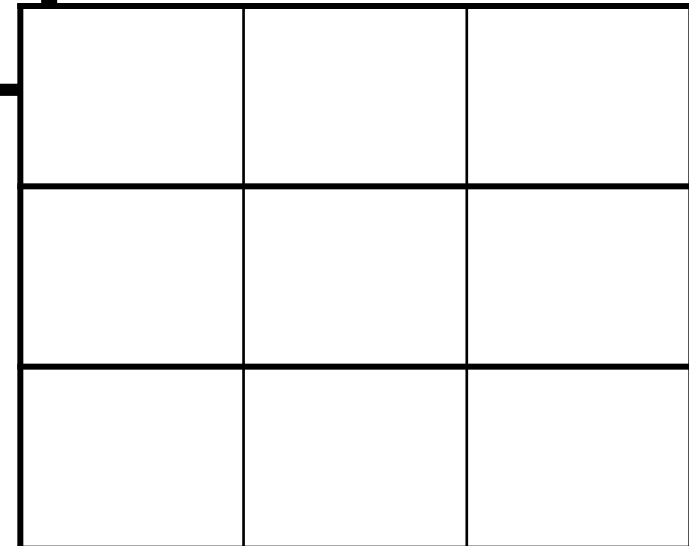
```
void hello() {  
    printf("Hello world\n");  
    return;  
}
```

```
int main() {  
    hello();  
    hello();  
    return 0;  
}
```

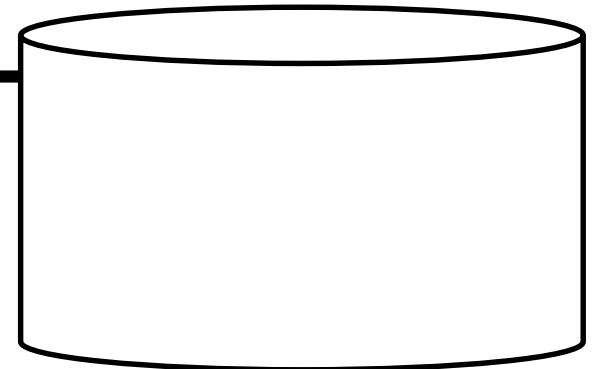
O main() fica parado aqui, aguardando a função hello() retornar

Hello world

Memória



HD



Execução

```
#include <stdio.h>
```

```
void hello() {
```

```
→ printf("Hello world\n");
```

```
    return;
```

```
}
```

```
int main() {
```

```
    hello();
```

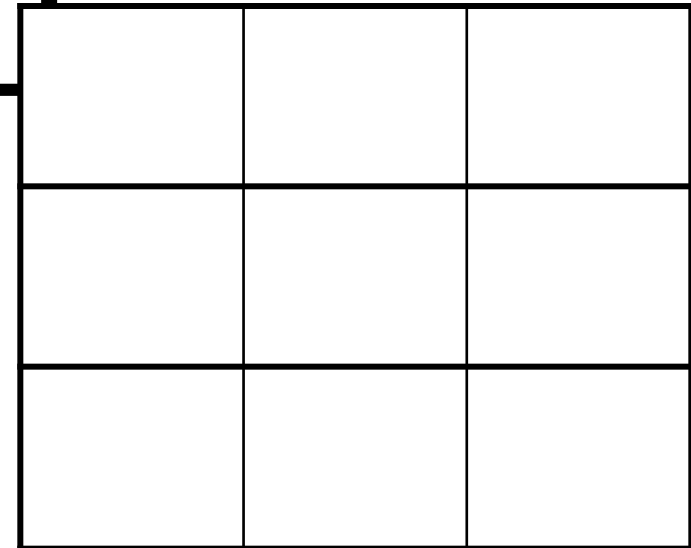
```
→ hello();
```

```
    return 0;
```

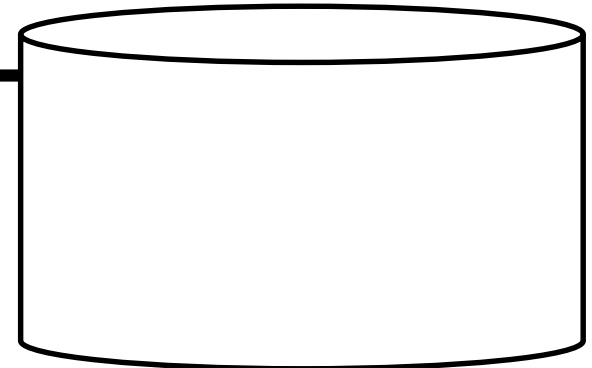
```
}
```

Hello world
Hello world

Memória



HD



Execução

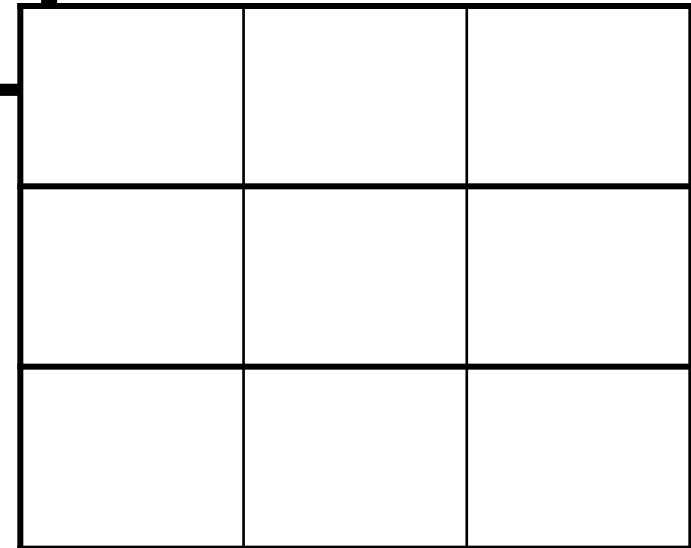
```
#include <stdio.h>

void hello() {
    printf("Hello world\n");
    return;
}

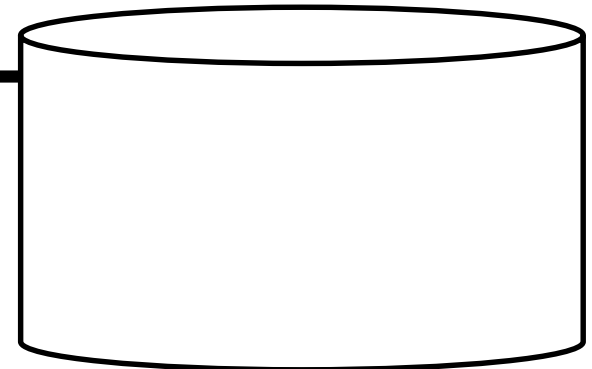
int main() {
    hello();
    hello();
    return 0;
}
```

Hello world
Hello world

Memória



HD



Execução

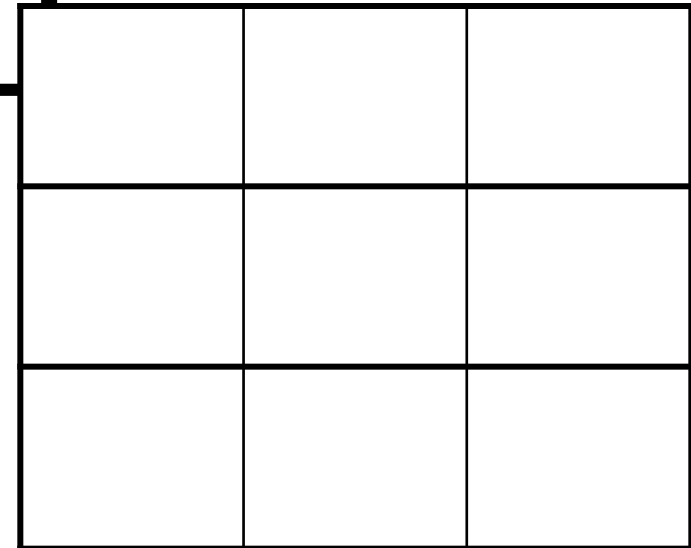
```
#include <stdio.h>

void hello() {
    printf("Hello world\n");
    return;
}

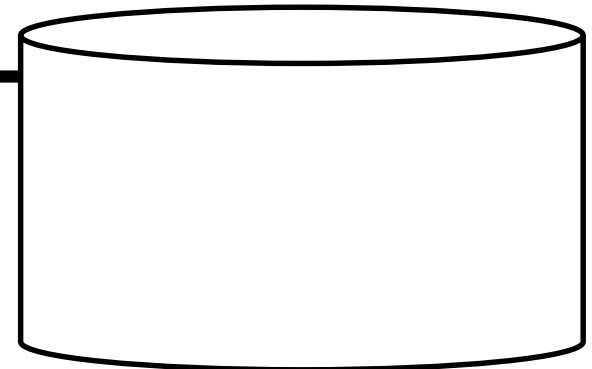
int main() {
    hello();
    → hello();
    return 0;
}
```

Hello world
Hello world

Memória



HD

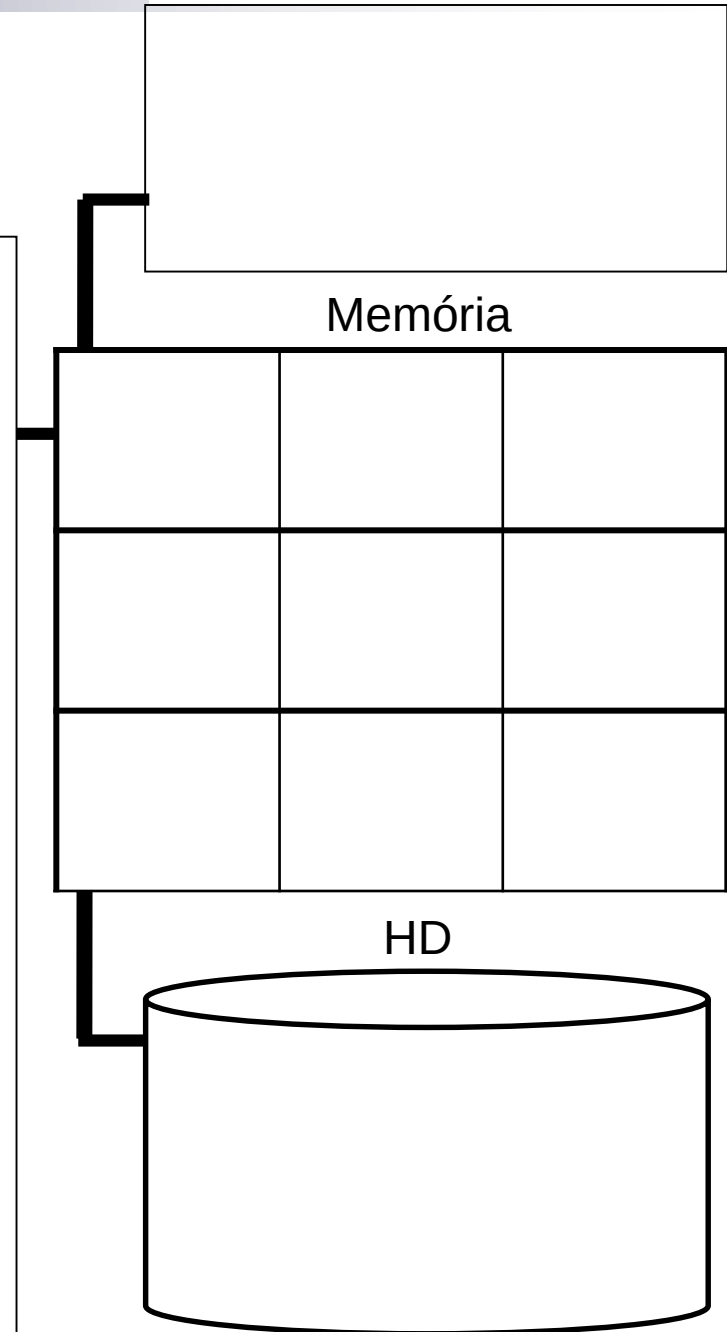
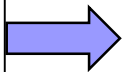


Execução

```
#include <stdio.h>

void hello() {
    printf("Hello world\n");
    return;
}

int main() {
    hello();
    hello();
    return 0;
}
```



Função

■ O que este programa faz?

```
#include <stdio.h>

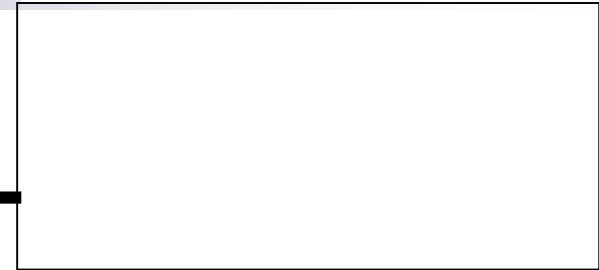
void alo() {
    printf("Alo Mundo\n");
}

void hello() {
    printf("Hello world\n");
    alo();
    printf("Fim\n");
}

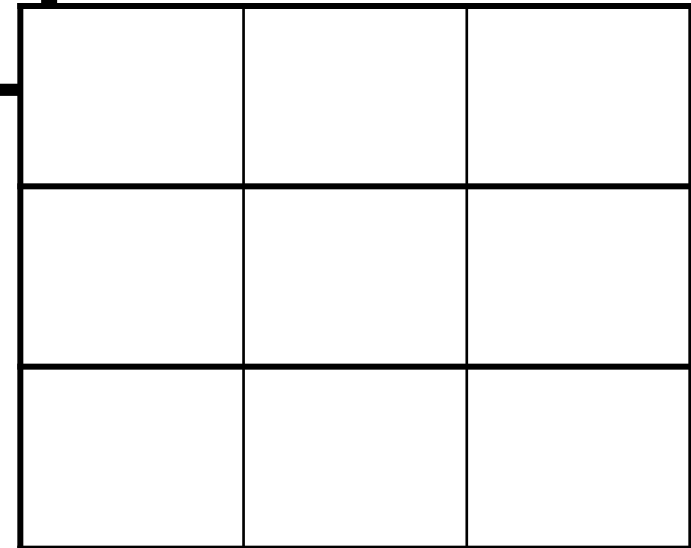
int main() {
    hello();
    return 0;
}
```

Execução

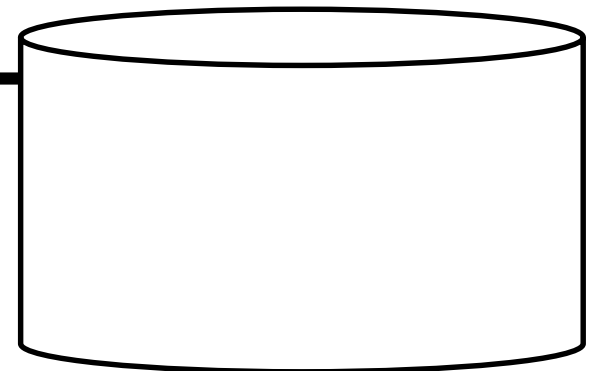
```
#include <stdio.h>
void alo() {
    printf("Alo Mundo\n");
}
void hello() {
    printf("Hello world\n");
    alo();
    printf("Fim\n");
}
→ int main() {
    hello();
    return 0;
}
```



Memória



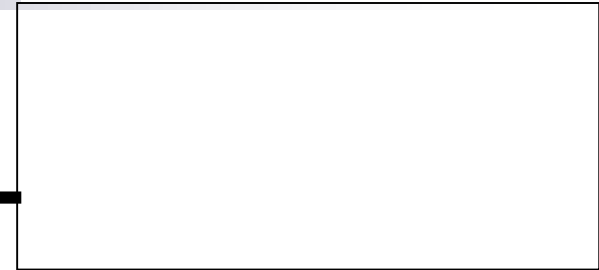
HD



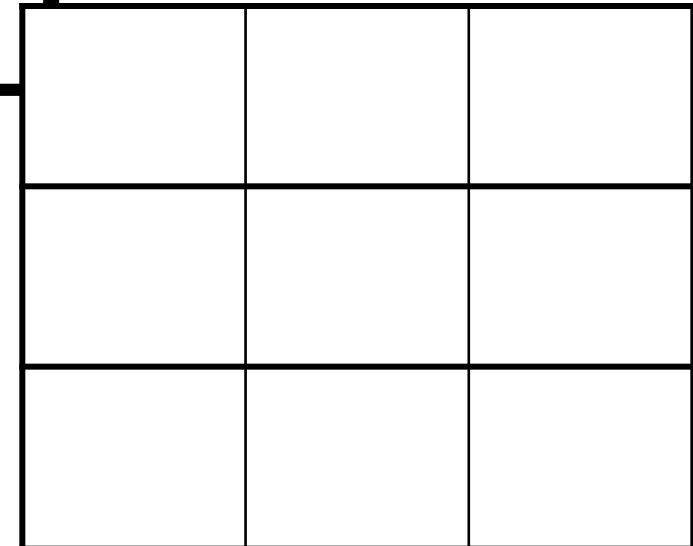
Execução

```
#include <stdio.h>
void alo() {
    printf("Alo Mundo\n");
}
void hello() {
    printf("Hello world\n");
    alo();
    printf("Fim\n");
}
int main() {
    hello();
    return 0;
}
```

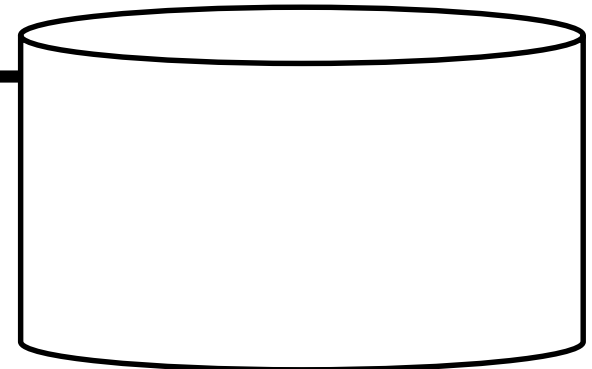
*O computador
chama hello() para
trabalhar*



Memória



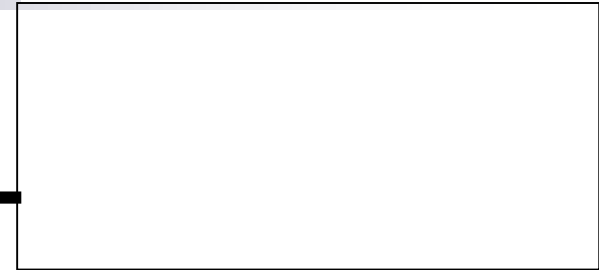
HD



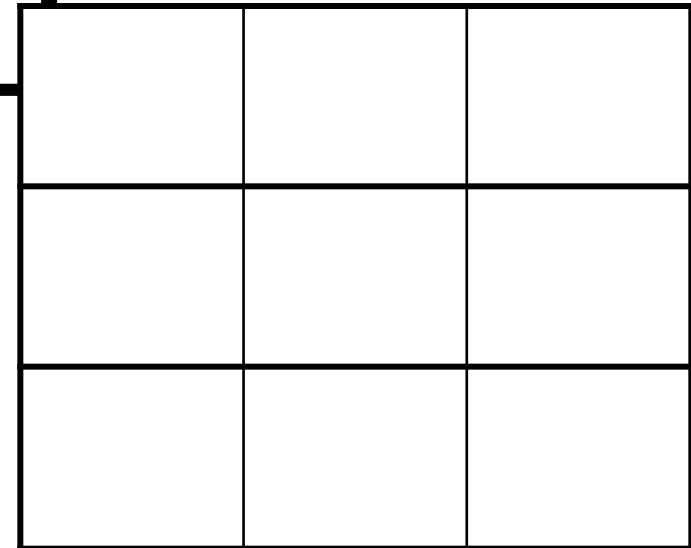
Execução

```
#include <stdio.h>
void alo() {
    printf("Alo Mundo\n");
}
void hello() {
    printf("Hello world\n");
    alo();
    printf("Fim\n");
}
int main() {
    hello();
    return 0;
}
```

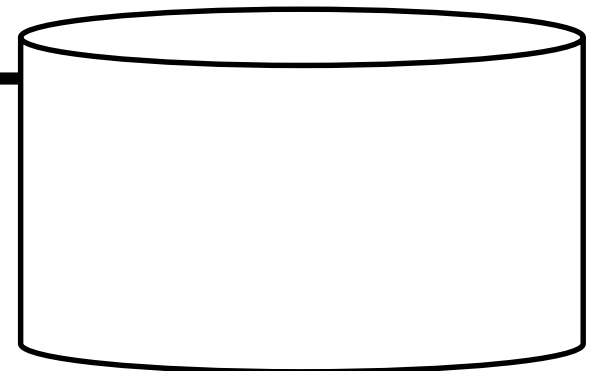
O main() fica parado aqui, aguardando a função hello() retornar



Memória



HD

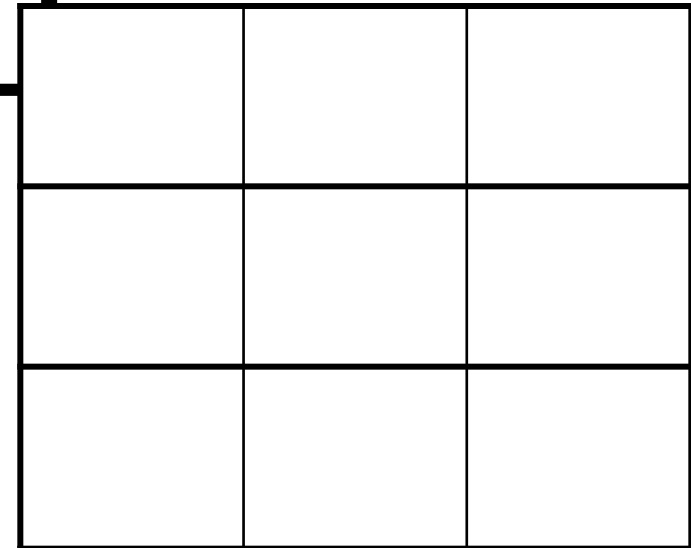


Execução

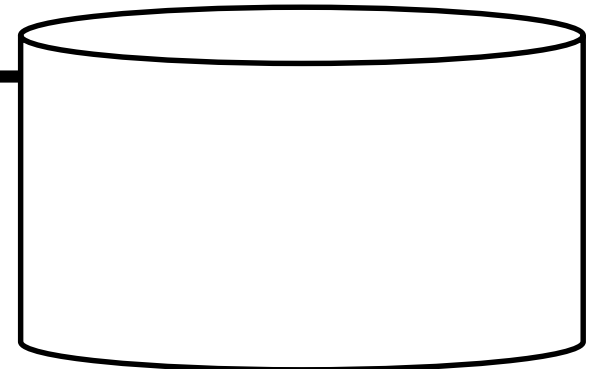
```
#include <stdio.h>
void alo() {
    printf("Alo Mundo\n");
}
void hello() {
    printf("Hello world\n");
    alo();
    printf("Fim\n");
}
int main() {
    hello();
    return 0;
}
```

Hello world

Memória



HD



Execução

```
#include <stdio.h>

void alo() {
    printf("Alo Mundo\n");
}

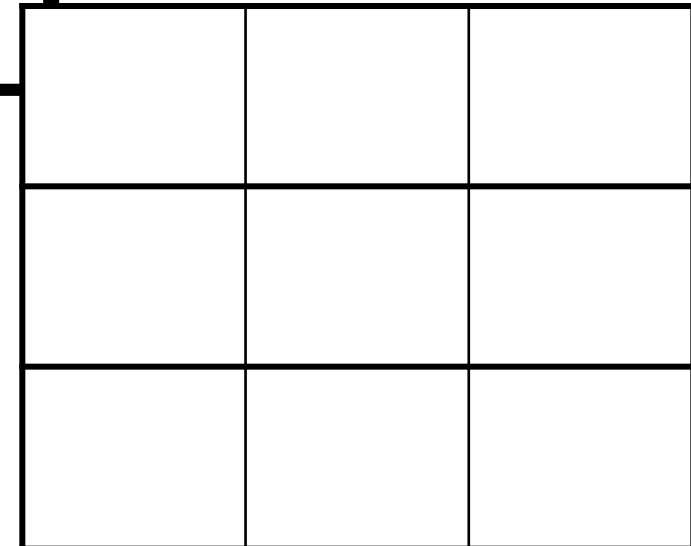
void hello() {
    printf("Hello world\n");
    alo();
    printf("Fim\n");
}

int main() {
    hello();
    return 0;
}
```

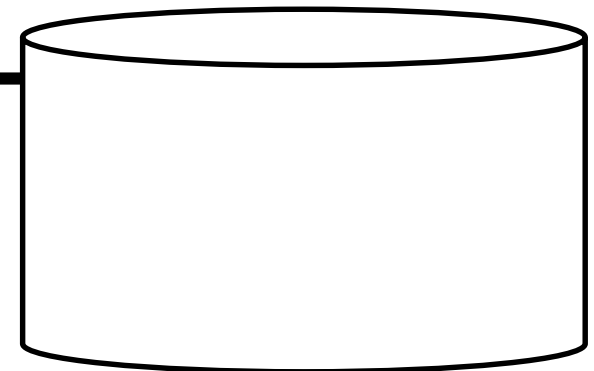
*O computador
chama alo() para
trabalhar*

Hello world

Memória



HD



Execução

```
#include <stdio.h>
```

```
void alo() {  
    printf("Alo Mundo\n");  
}
```

```
void hello() {  
    printf("Hello world\n");
```

```
    alo();  
    printf("Fim\n");  
}
```

```
int main() {
```

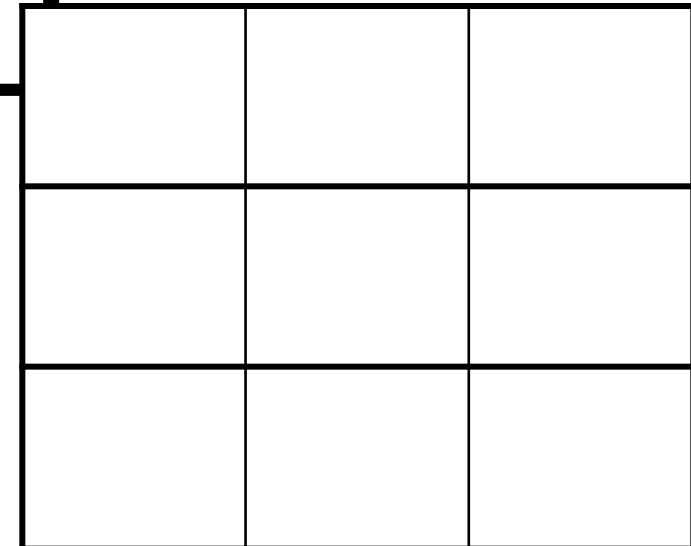
```
    hello();  
    return 0;
```

```
}
```

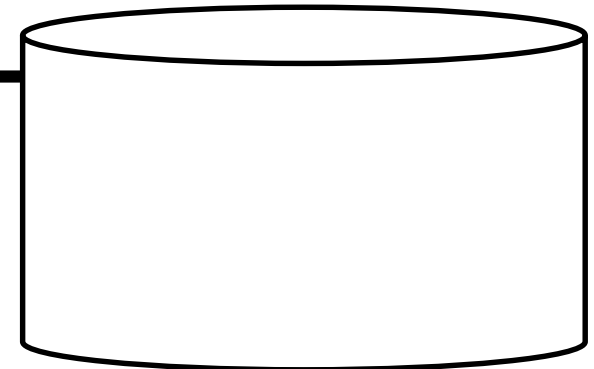
O hello() fica parado aqui, aguardando a função alo() retornar

Hello world

Memória



HD

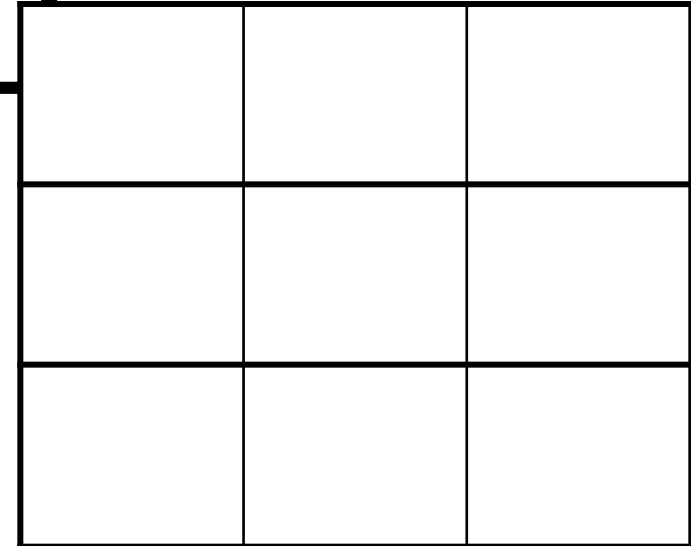


Execução

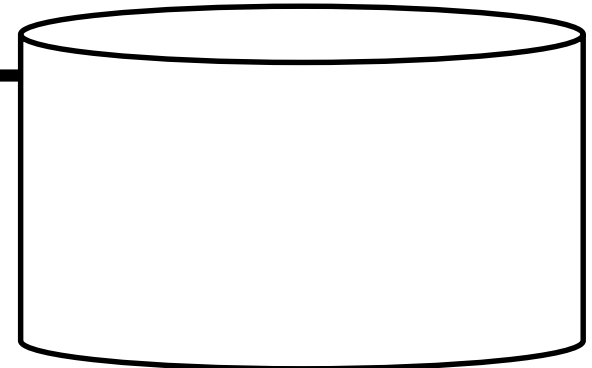
```
#include <stdio.h>
void alo() {
    printf("Alo Mundo\n");
}
void hello() {
    printf("Hello world\n");
    alo();
    printf("Fim\n");
}
int main() {
    hello();
    return 0;
}
```

Hello world
Alo Mundo

Memória



HD

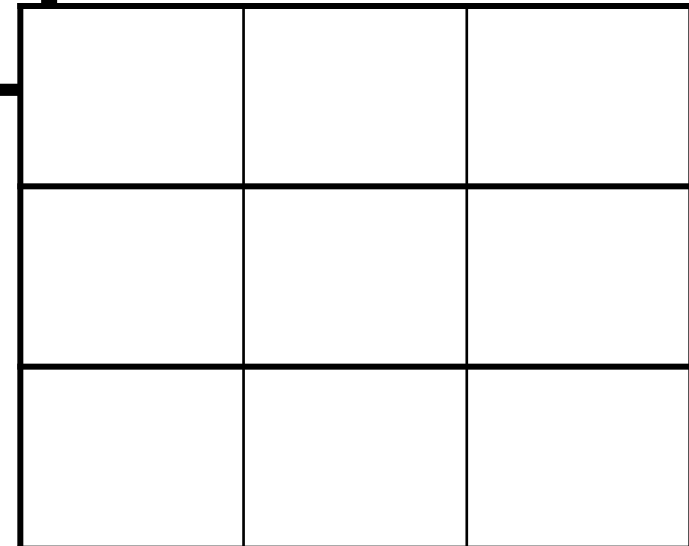


Execução

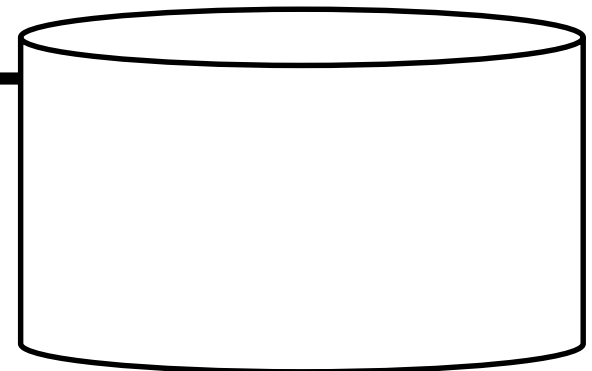
```
#include <stdio.h>
void alo() {
    printf("Alo Mundo\n");
}
void hello() {
    printf("Hello world\n");
    alo();
    printf("Fim\n");
}
int main() {
    hello();
    return 0;
}
```

Hello world
Alo Mundo

Memória



HD

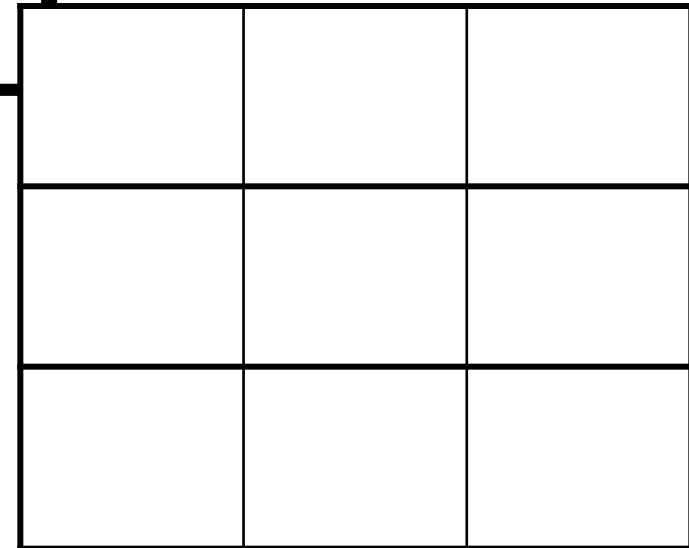


Execução

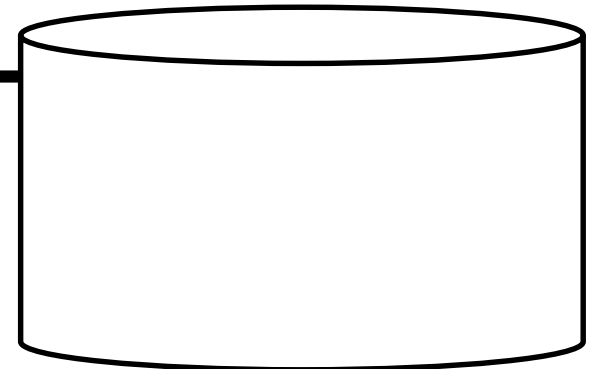
```
#include <stdio.h>
void alo() {
    printf("Alo Mundo\n");
}
void hello() {
    printf("Hello world\n");
    alo();
    printf("Fim\n");
}
int main() {
    hello();
    return 0;
}
```

Hello world
Alo Mundo
Fim

Memória



HD



Execução

```
#include <stdio.h>

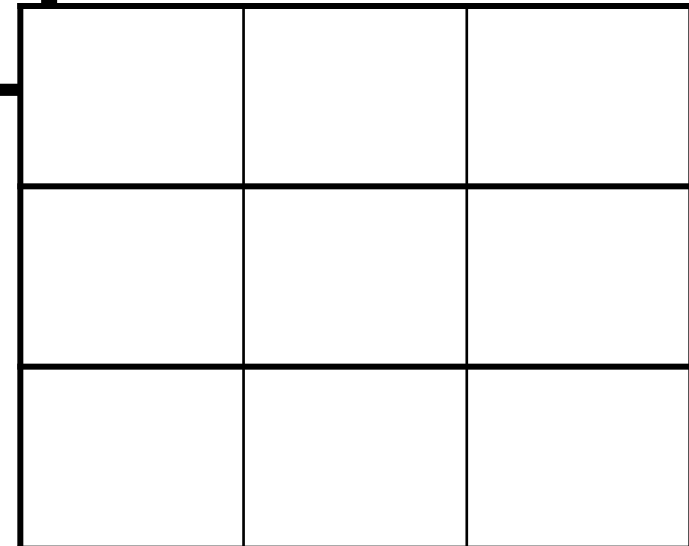
void alo() {
    printf("Alo Mundo\n");
}

void hello() {
    printf("Hello world\n");
    alo();
    printf("Fim\n");
}

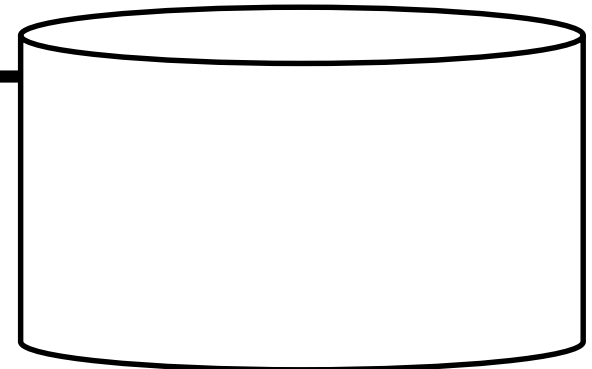
int main() {
    hello();
    return 0;
}
```

Hello world
Alo Mundo
Fim

Memória



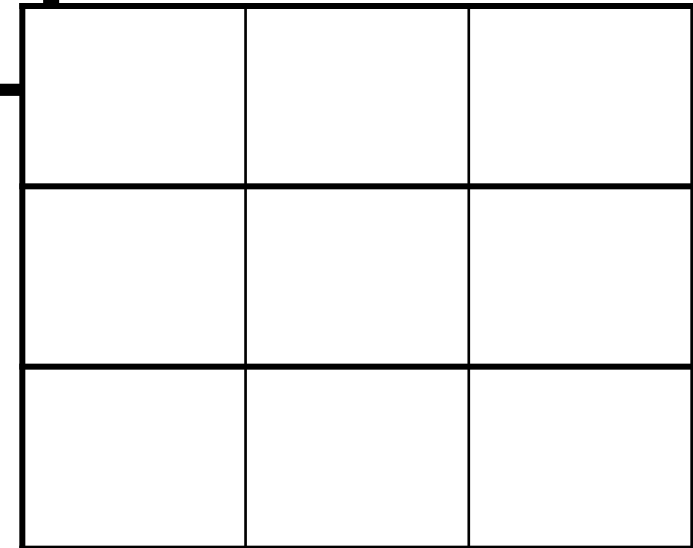
HD



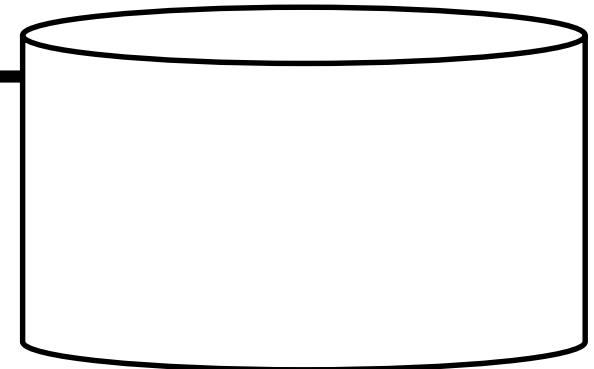
Execução

```
#include <stdio.h>
void alo() {
    printf("Alo Mundo\n");
}
void hello() {
    printf("Hello world\n");
    alo();
    printf("Fim\n");
}
int main() {
    hello();
    return 0;
}
```

Memória



HD



Definição de Função

- Antes da função `main()`
 - O computador precisa saber de antemão todas funções que existem
- Caso ocorram chamadas de funções por outras funções, as funções a serem chamadas precisam ser definidas antes

Definição de Função

```
#include <stdio.h>
```

```
void hello() {  
    printf("Hello world\n");  
}
```

Definição de hello()

```
int main() {
```

```
    hello();
```

Uso de hello()

```
    return 0;
```

```
}
```

Antes de usarmos a função hello(), precisamos dizer ao computador o que ela recebe como argumento, o que ela faz e o que ela retorna.

Definição de Função

Opcional

- Se você quiser, a função pode ser definida após o uso
- Porém, deve-se declarar ao menos seu nome e parâmetros antes do uso
- Tal declaração chama-se *assinatura*, *interface* ou *protótipo*

Função

```
#include <stdio.h>
```

```
int dobro(int x);
```

```
int main() {  
    int n,d;  
    printf("Entre n: ");  
    scanf("%d", &n);  
    d = dobro(n);  
    printf("%d\n",d);  
    return 0;  
}
```

```
int dobro(int x) {  
    x = 2 * x;  
    return x;  
}
```

*Assinatura da função.
Assim, a definição da
função não precisa vir
antes do seu uso.*



Esta facilidade é opcional.

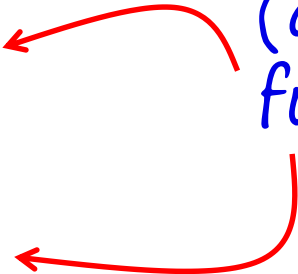
Função

```
#include<stdio.h>
int dobro(int x);
```

```
int dobro(int x) {
    x = 2 * x;
    return x;
}
```

```
int main() {
    int n,d;
    printf("Entre n: ");
    scanf("%d", &n);
    d = dobro(n);
    printf("%d\n",d);
    return 0;
}
```

*Também é permitido
(desnecessário, mas
funciona).*



Escopo de Variáveis

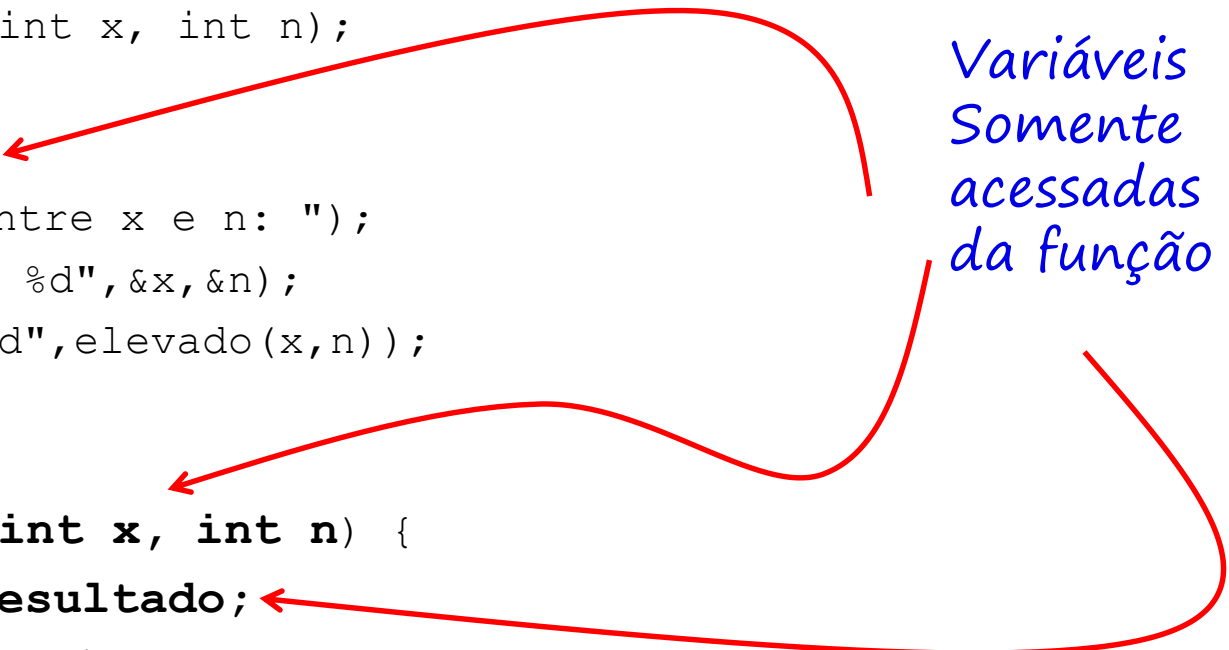
- Escopo de um variável é a região do programa, na qual uma variável pode ser utilizada
- **Variáveis Locais.** Declarada dentro de uma função e só pode ser utilizada pela função que a declarou. Ao término da função, a variável é apagada.
- **Variáveis Globais.** Declarada fora das funções e pode ser utilizada em qualquer parte do programa. Só é apagada quando o main() termina.

Escopo de Variáveis

```
#include <stdio.h>
```

```
int elevado(int x, int n);  
int main() {  
    int x,n;  
    printf("Entre x e n: ");  
    scanf("%d %d",&x,&n);  
    printf("%d",elevado(x,n));  
    return 0;  
}  
int elevado(int x, int n) {  
    int i, resultado;  
    resultado = 1;  
    for(i = 0; i < n; i++) {  
        resultado = x * resultado;  
    }  
    return resultado;  
}
```

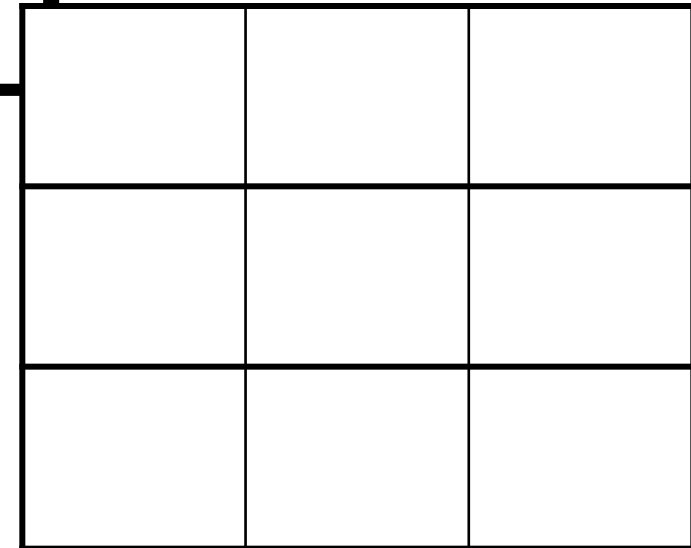
Variáveis locais.
Somente são
acessadas dentro
da função



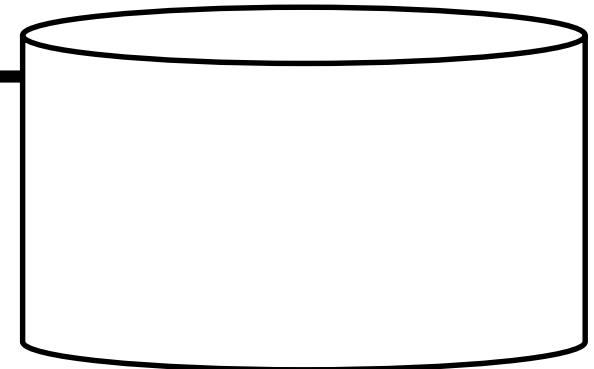
Execução

```
#include <stdio.h>
int elevado(int x, int n);
int main() {
    int x,n;
    printf("Entre x e n: ");
    scanf("%d %d",&x,&n);
    printf("%d",elevado(x,n));
    return 0;
}
int elevado(int x, int n) {
    int i, resultado;
    resultado = 1;
    for(i = 0; i < n; i++) {
        resultado = x * resultado;
    }
    return resultado;
}
```

Memória

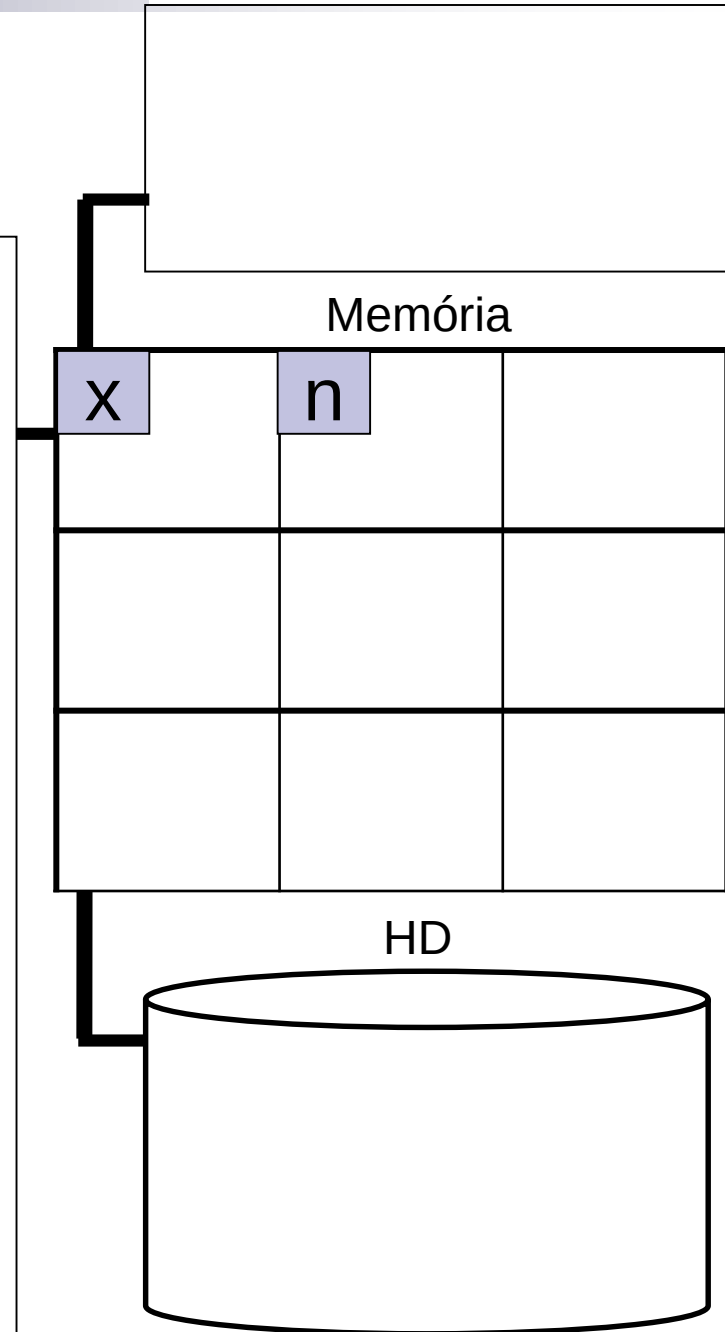


HD



Execução

```
#include <stdio.h>
int elevado(int x, int n);
int main() {
    int x,n;
    printf("Entre x e n: ");
    scanf("%d %d",&x,&n);
    printf("%d",elevado(x,n));
    return 0;
}
int elevado(int x, int n) {
    int i, resultado;
    resultado = 1;
    for(i = 0; i < n; i++) {
        resultado = x * resultado;
    }
    return resultado;
}
```



Execução

```
#include <stdio.h>
int elevado(int x, int n);
int main() {
    int x,n;
    printf("Entre x e n: ");
    scanf("%d %d",&x,&n);
    printf("%d",elevado(x,n));
    return 0;
}
int elevado(int x, int n) {
    int i, resultado;
    resultado = 1;
    for(i = 0; i < n; i++) {
        resultado = x * resultado;
    }
    return resultado;
}
```

Entre x e n:

Memória

x

n

HD

Execução

```
#include <stdio.h>
int elevado(int x, int n);
int main() {
    int x,n;
    printf("Entre x e n: ");
    scanf("%d %d",&x,&n);
    printf("%d",elevado(x,n));
    return 0;
}
int elevado(int x, int n) {
    int i, resultado;
    resultado = 1;
    for(i = 0; i < n; i++) {
        resultado = x * resultado;
    }
    return resultado;
}
```

Entre x e n:

Memória

x

n

HD

Execução

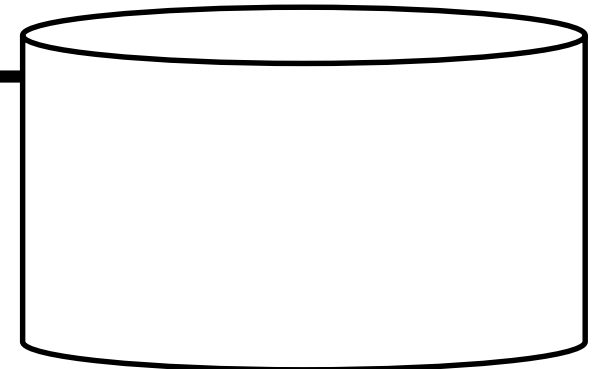
```
#include <stdio.h>
int elevado(int x, int n);
int main() {
    int x,n;
    printf("Entre x e n: ");
    scanf("%d %d",&x,&n);
    printf("%d",elevado(x,n));
    return 0;
}
int elevado(int x, int n) {
    int i, resultado;
    resultado = 1;
    for(i = 0; i < n; i++) {
        resultado = x * resultado;
    }
    return resultado;
}
```

Entre x e n: 2 3

Memória

x	n	

HD



Entre x e n: 2 3 <ENTER>

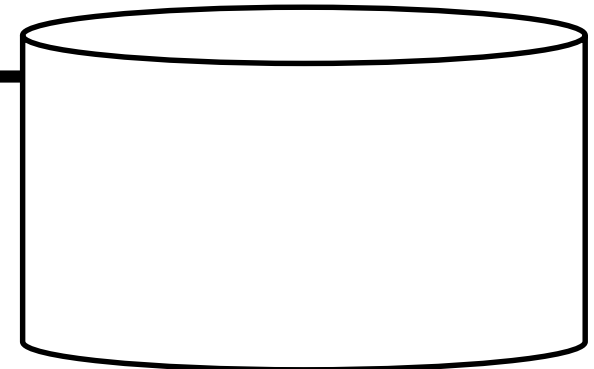
Execução

```
#include <stdio.h>
int elevado(int x, int n);
int main() {
    int x,n;
    printf("Entre x e n: ");
    scanf("%d %d",&x,&n);
    printf("%d",elevado(x,n));
    return 0;
}
int elevado(int x, int n) {
    int i, resultado;
    resultado = 1;
    for(i = 0; i < n; i++) {
        resultado = x * resultado;
    }
    return resultado;
}
```

Memória

x	2	n	3	

HD



Entre x e n: 2 3 <ENTER>

Execução

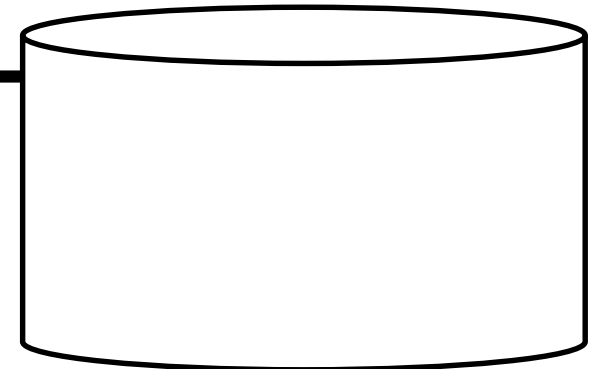
```
#include <stdio.h>
int elevado(int x, int n);
int main() {
    int x,n;
    printf("Entre x e n: ");
    scanf("%d %d",&x,&n);
    printf("%d",elevado(x,n));
    return 0;
}
int elevado(int x, int n) {
    int i, resultado;
    resultado = 1;
    for(i = 0; i < n; i++) {
        resultado = x * resultado;
    }
    return resultado;
}
```

*Antes de fazer o
printf(), o
computador
chama
elevado(2,3) para
trabalhar.*

Memória

x	2	n	3	

HD



Entre x e n: 2 3 <ENTER>

Execução

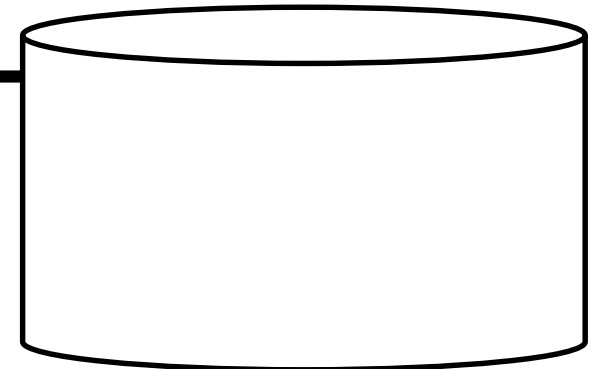
```
#include <stdio.h>
int elevado(int x, int n);
int main() {
    int x,n;
    printf("Entre x e n: ");
    scanf("%d %d",&x,&n);
    printf("%d",elevado(x,n));
    return 0;
}
int elevado(int x, int n) {
    int i, resultado;
    resultado = 1;
    for(i = 0; i < n; i++) {
        resultado = x * resultado;
    }
    return resultado;
}
```

O main() fica parado aqui, aguardando a função elevado() retornar.

Memória

x	2	n	3

HD



Entre x e n: 2 3 <ENTER>

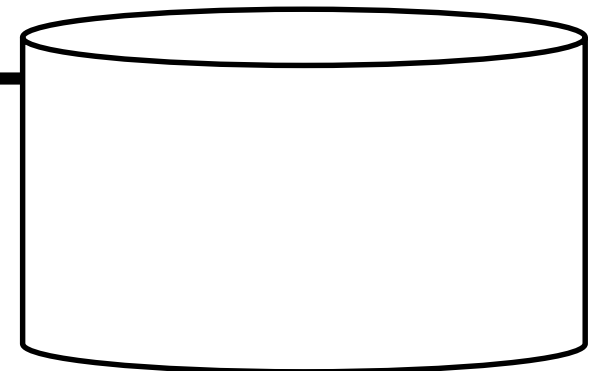
Execução

```
#include <stdio.h>
int elevado(int x, int n);
int main() {
    int x,n;
    printf("Entre x e n: ");
    scanf("%d %d",&x,&n);
    printf("%d",elevado(x,n));
    return 0;
}
int elevado(int x, int n) {
    int i, resultado;
    resultado = 1;
    for(i = 0; i < n; i++) {
        resultado = x * resultado;
    }
    return resultado;
}
```

Memória

x	2	n	3	
x _{elevado()}	2	n _{elevado()}	3	

HD



Entre x e n: 2 3 <ENTER>

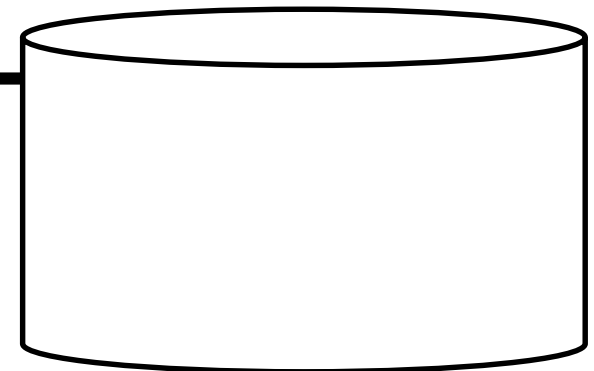
Execução

```
#include <stdio.h>
int elevado(int x, int n);
int main() {
    int x,n;
    printf("Entre x e n: ");
    scanf("%d %d",&x,&n);
    printf("%d",elevado(x,n));
    return 0;
}
int elevado(int x, int n) {
    int i, resultado;
    resultado = 1;
    for(i = 0; i < n; i++) {
        resultado = x * resultado;
    }
    return resultado;
}
```

Memória

x	2	n	3	
x _{elevado()}		n _{elevado()}		
	2		3	
i _{elevado()}		resultado _{elevado()}		

HD



Entre x e n: 2 3 <ENTER>

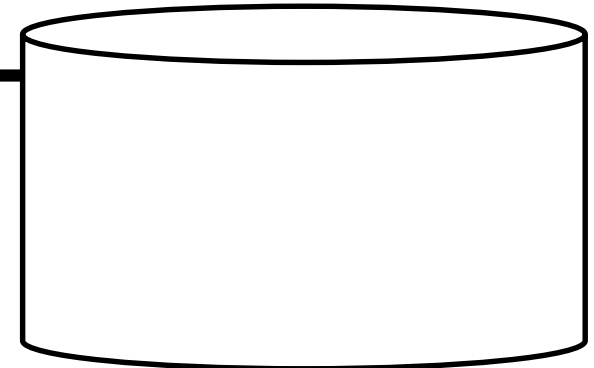
Execução

```
#include <stdio.h>
int elevado(int x, int n);
int main() {
    int x,n;
    printf("Entre x e n: ");
    scanf("%d %d",&x,&n);
    printf("%d",elevado(x,n));
    return 0;
}
int elevado(int x, int n) {
    int i, resultado;
    resultado = 1;
    for(i = 0; i < n; i++) {
        resultado = x * resultado;
    }
    return resultado;
}
```

Memória

x	n	
2	3	
x _{elevado()}	n _{elevado()}	
2	3	
i _{elevado()}	resultado _{elevado()}	
	1	

HD



Entre x e n: 2 3 <ENTER>

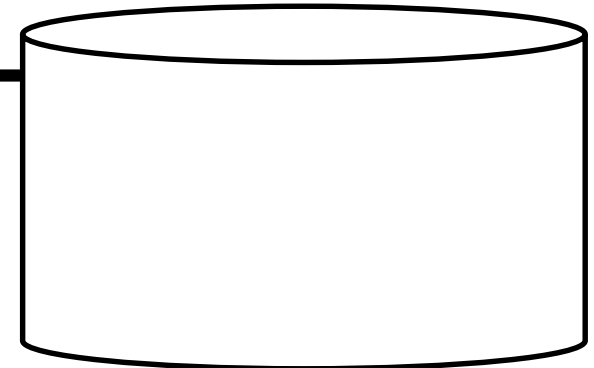
Execução

```
#include <stdio.h>
int elevado(int x, int n);
int main() {
    int x,n;
    printf("Entre x e n: ");
    scanf("%d %d",&x,&n);
    printf("%d",elevado(x,n));
    return 0;
}
int elevado(int x, int n) {
    int i, resultado;
    resultado = 1;
    for(i = 0; i < n; i++) {
        resultado = x * resultado;
    }
    return resultado;
}
```

Memória

x	n	
2	3	
x _{elevado()}	n _{elevado()}	
2	3	
i _{elevado()}	resultado _{elevado()}	
0	1	

HD



Entre x e n: 2 3 <ENTER>

Execução

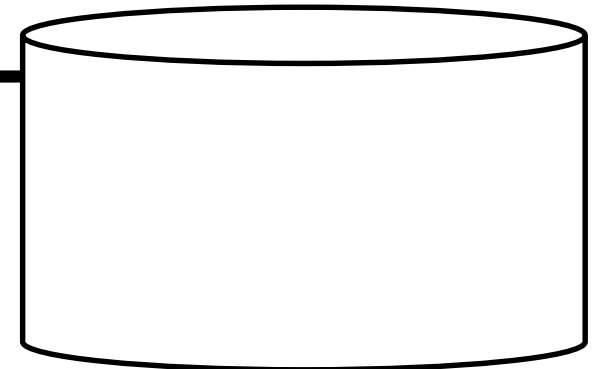
```
#include <stdio.h>
int elevado(int x, int n);
int main() {
    int x,n;
    printf("Entre x e n: ");
    scanf("%d %d",&x,&n);
    printf("%d",elevado(x,n));
    return 0;
}
int elevado(int x, int n) {
    int i, resultado;
    resultado = 1;
    for(i = 0; i < n; i++) {
        resultado = x * resultado;
    }
    return resultado;
}
```

$i_{elevado()} < n_{elevado()}$
 $= 0 < 3$
 $= 1$

Memória

x	n	
2	3	
x _{elevado()}	n _{elevado()}	
2	3	
i _{elevado()}	resultado _{elevado()}	
0	1	

HD



Entre x e n: 2 3 <ENTER>

Execução

```
#include <stdio.h>
int elevado(int x, int n);
int main() {
    int x,n;
    printf("Entre x e n: ");
    scanf("%d %d",&x,&n);
    printf("%d",elevado(x,n));
    return 0;
}
```

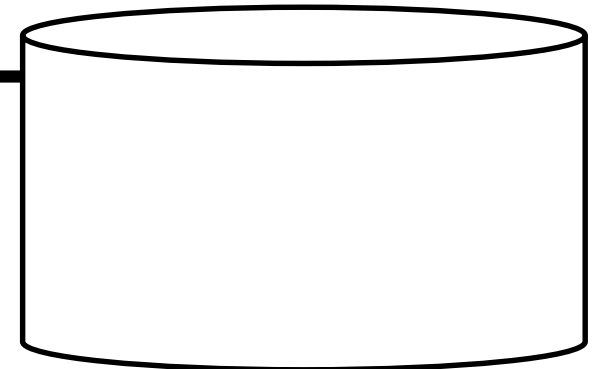
```
int elevado(int x, int n) {
    int i, resultado;
    resultado = 1;
    for(i = 0; i < n; i++) {
        resultado = x * resultado;
    }
    return resultado;
}
```

$x_{elevado()} * resultado_{elevado()}$
 $= 2 * 1$
 $= 2$

Memória

x	2	n	3	
$x_{elevado()}$		$n_{elevado()}$		
	2		3	
$i_{elevado()}$		resultado _{elevado()}		
	0		1	

HD



Entre x e n: 2 3 <ENTER>

Execução

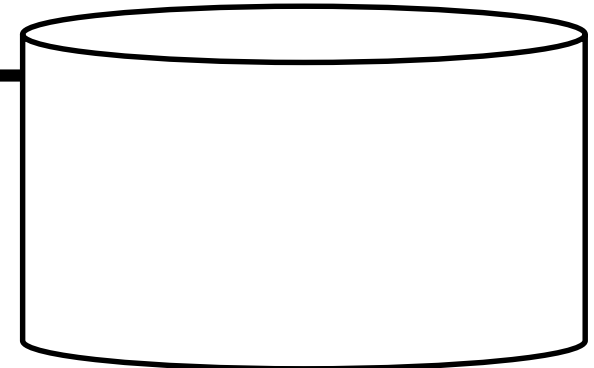
```
#include <stdio.h>
int elevado(int x, int n);
int main() {
    int x,n;
    printf("Entre x e n: ");
    scanf("%d %d",&x,&n);
    printf("%d",elevado(x,n));
    return 0;
}
int elevado(int x, int n) {
    int i, resultado;
    resultado = 1;
    for(i = 0; i < n; i++) {
        resultado = x * resultado;
    }
    return resultado;
}
```

$x_{elevado()} * resultado_{elevado()}$
 $= 2 * 1$
 $= 2$

Memória

x	2	n	3	
$x_{elevado()}$		$n_{elevado()}$		
	2		3	
$i_{elevado()}$		$resultado_{elevado()}$		
	0		2	

HD



Entre x e n: 2 3 <ENTER>

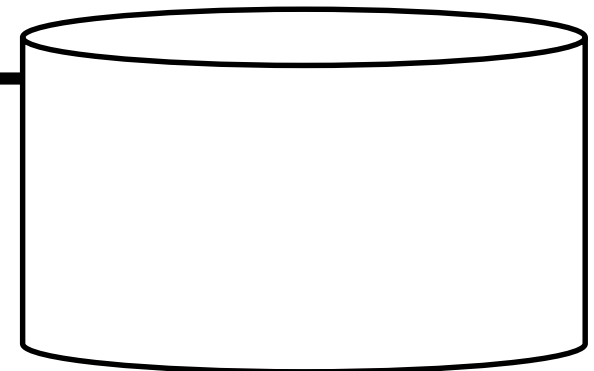
Execução

```
#include <stdio.h>
int elevado(int x, int n);
int main() {
    int x,n;
    printf("Entre x e n: ");
    scanf("%d %d",&x,&n);
    printf("%d",elevado(x,n));
    return 0;
}
int elevado(int x, int n) {
    int i, resultado;
    resultado = 1;
    for(i = 0; i < n; i++) {
        resultado = x * resultado;
    }
    return resultado;
}
```

Memória

x	2	n	3	
x _{elevado()}	2	n _{elevado()}	3	
i _{elevado()}	1	resultado _{elevado()}	2	

HD



Entre x e n: 2 3 <ENTER>

Execução

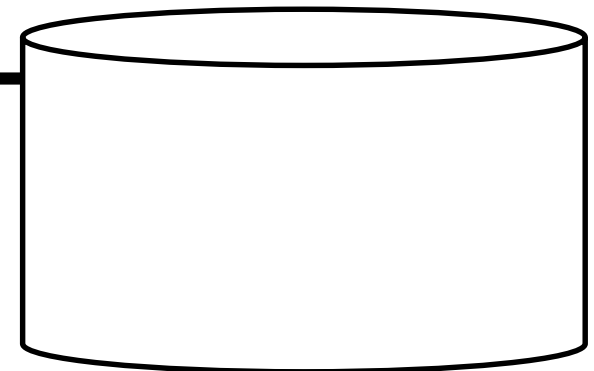
```
#include <stdio.h>
int elevado(int x, int n);
int main() {
    int x,n;
    printf("Entre x e n: ");
    scanf("%d %d",&x,&n);
    printf("%d",elevado(x,n));
    return 0;
}
int elevado(int x, int n) {
    int i, resultado;
    resultado = 1;
    for(i = 0; i < n; i++) {
        resultado = x * resultado;
    }
    return resultado;
}
```

$i_{elevado()} < n_{elevado()}$
 $= 1 < 3$
 $= 1$

Memória

x	n	
2	3	
$x_{elevado()}$	$n_{elevado()}$	
2	3	
$i_{elevado()}$	$resultado_{elevado()}$	
1	2	

HD



Entre x e n: 2 3 <ENTER>

Execução

```
#include <stdio.h>
int elevado(int x, int n);
int main() {
    int x,n;
    printf("Entre x e n: ");
    scanf("%d %d",&x,&n);
    printf("%d",elevado(x,n));
    return 0;
}
```

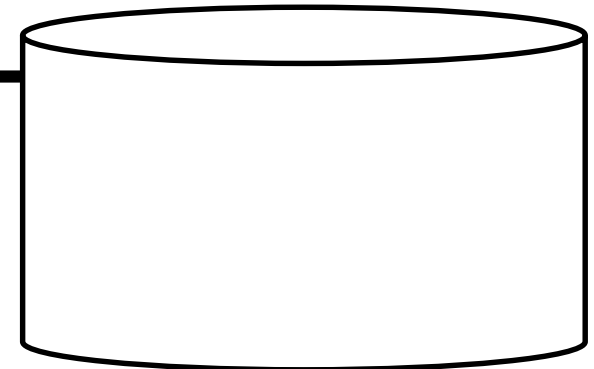
```
int elevado(int x, int n) {
    int i, resultado;
    resultado = 1;
    for(i = 0; i < n; i++) {
        resultado = x * resultado;
    }
    return resultado;
}
```

$x_{elevado()} * resultado_{elevado()}$
 $= 2 * 2$
 $= 4$

Memória

x	2	n	3	
$x_{elevado()}$		$n_{elevado()}$		
	2		3	
$i_{elevado()}$		$resultado_{elevado()}$		
	1		2	

HD



Entre x e n: 2 3 <ENTER>

Execução

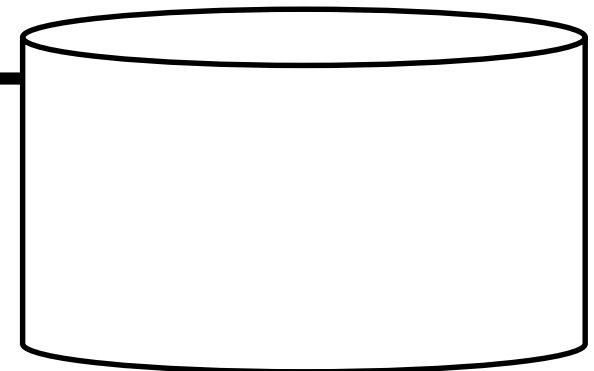
```
#include <stdio.h>
int elevado(int x, int n);
int main() {
    int x,n;
    printf("Entre x e n: ");
    scanf("%d %d",&x,&n);
    printf("%d",elevado(x,n));
    return 0;
}
int elevado(int x, int n) {
    int i, resultado;
    resultado = 1;
    for(i = 0; i < n; i++) {
        resultado = x * resultado;
    }
    return resultado;
}
```

$x_{elevado()} * resultado_{elevado()}$
 $= 2 * 2$
 $= 4$

Memória

x	2	n	3	
$x_{elevado()}$		$n_{elevado()}$		
	2		3	
$i_{elevado()}$		resultado _{elevado()}		
	1		4	

HD



Entre x e n: 2 3 <ENTER>

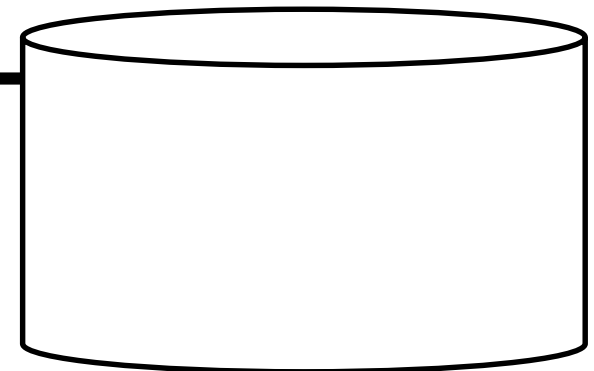
Execução

```
#include <stdio.h>
int elevado(int x, int n);
int main() {
    int x,n;
    printf("Entre x e n: ");
    scanf("%d %d",&x,&n);
    printf("%d",elevado(x,n));
    return 0;
}
int elevado(int x, int n) {
    int i, resultado;
    resultado = 1;
    for(i = 0; i < n; i++) {
        resultado = x * resultado;
    }
    return resultado;
}
```

Memória

x	n	
2	3	
x _{elevado()}	n _{elevado()}	
2	3	
i _{elevado()}	resultado _{elevado()}	
2	4	

HD



Entre x e n: 2 3 <ENTER>

Execução

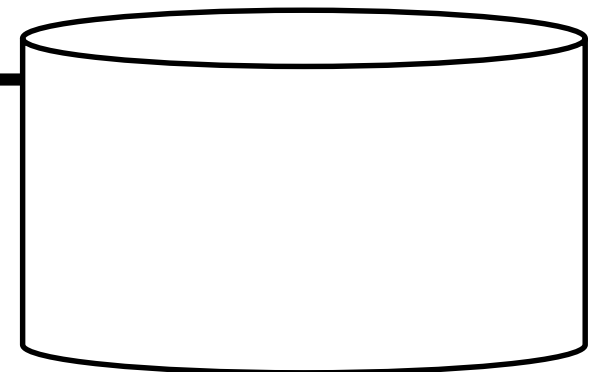
```
#include <stdio.h>
int elevado(int x, int n);
int main() {
    int x,n;
    printf("Entre x e n: ");
    scanf("%d %d",&x,&n);
    printf("%d",elevado(x,n));
    return 0;
}
int elevado(int x, int n) {
    int i, resultado;
    resultado = 1;
    for(i = 0; i < n; i++) {
        resultado = x * resultado;
    }
    return resultado;
}
```

$i_{elevado()} < n_{elevado()}$
 $= 2 < 3$
 $= 1$

Memória

x	n	
2	3	
$x_{elevado()}$	$n_{elevado()}$	
2	3	
$i_{elevado()}$	$resultado_{elevado()}$	
2	4	

HD



Entre x e n: 2 3 <ENTER>

Execução

```
#include <stdio.h>
int elevado(int x, int n);
int main() {
    int x,n;
    printf("Entre x e n: ");
    scanf("%d %d",&x,&n);
    printf("%d",elevado(x,n));
    return 0;
```

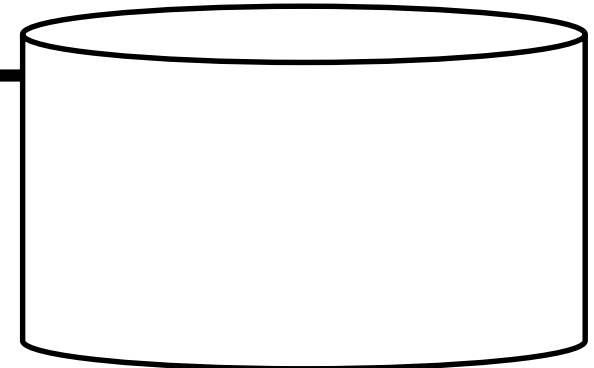
```
}
int elevado(int x, int n) {
    int i, resultado;
    resultado = 1;
    for(i = 0; i < n; i++) {
        resultado = x * resultado;
    }
    return resultado;
}
```

$x_{elevado()} * resultado_{elevado()}$
 $= 2 * 4$
 $= 8$

Memória

x	2	n	3	
$x_{elevado()}$		$n_{elevado()}$		
	2		3	
$i_{elevado()}$		resultado _{elevado()}		
	2		4	

HD



Entre x e n: 2 3 <ENTER>

Execução

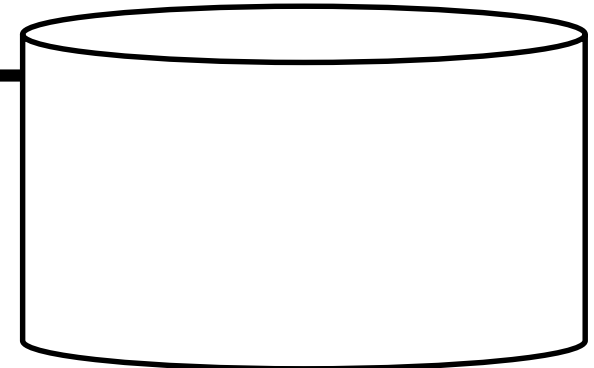
```
#include <stdio.h>
int elevado(int x, int n);
int main() {
    int x,n;
    printf("Entre x e n: ");
    scanf("%d %d",&x,&n);
    printf("%d",elevado(x,n));
    return 0;
}
int elevado(int x, int n) {
    int i, resultado;
    resultado = 1;
    for(i = 0; i < n; i++) {
        resultado = x * resultado;
    }
    return resultado;
}
```

$x_{elevado()} * resultado_{elevado()}$
 $= 2 * 4$
 $= 8$

Memória

x	2	n	3	
$x_{elevado()}$		$n_{elevado()}$		
	2		3	
$i_{elevado()}$		$resultado_{elevado()}$		
	2	8		

HD



Entre x e n: 2 3 <ENTER>

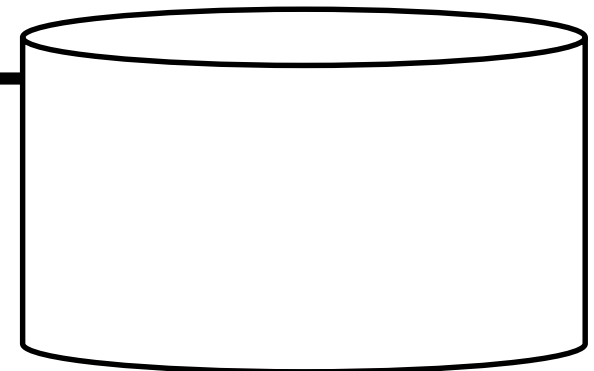
Execução

```
#include <stdio.h>
int elevado(int x, int n);
int main() {
    int x,n;
    printf("Entre x e n: ");
    scanf("%d %d",&x,&n);
    printf("%d",elevado(x,n));
    return 0;
}
int elevado(int x, int n) {
    int i, resultado;
    resultado = 1;
    for(i = 0; i < n; i++) {
        resultado = x * resultado;
    }
    return resultado;
}
```

Memória

x	2	n	3	
x _{elevado()}	2	n _{elevado()}	3	
i _{elevado()}	3	resultado _{elevado()}	8	

HD



Entre x e n: 2 3 <ENTER>

Execução

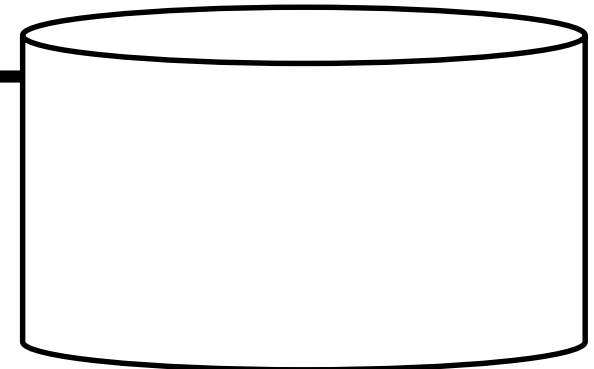
```
#include <stdio.h>
int elevado(int x, int n);
int main() {
    int x,n;
    printf("Entre x e n: ");
    scanf("%d %d",&x,&n);
    printf("%d",elevado(x,n));
    return 0;
}
int elevado(int x, int n) {
    int i, resultado;
    resultado = 1;
    for(i = 0; i < n; i++) {
        resultado = x * resultado;
    }
    return resultado;
}
```

$i_{elevado()} < n_{elevado()}$
 $= 3 < 3$
 $= 0$

Memória

x	n	
2	3	
x _{elevado()}	n _{elevado()}	
2	3	
i _{elevado()}	resultado _{elevado()}	
3	8	

HD



Entre x e n: 2 3 <ENTER>

Execução

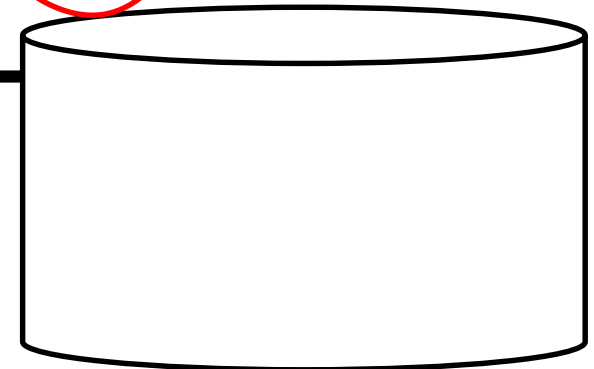
```
#include <stdio.h>
int elevado(int x, int n);
int main() {
    int x,n;
    printf("Entre x e n: ");
    scanf("%d %d",&x,&n);
    printf("%d",elevado(x,n));
    return 0;
}

int elevado(int x, int n) {
    int i, resultado;
    resultado = 1;
    for(i = 0; i < n; i++) {
        resultado = x * resultado;
    }
    return resultado;
}
```

Memória

x	2	n	3	
x _{elevado()}	2	n _{elevado()}	3	
i _{elevado()}	3	resultado _{elevado()}	8	

HD



O valor de resultado_{elevado()} é retornado ao main() no lugar da chamada de elevado().

Execução

```
#include <stdio.h>
int elevado(int x, int n);
int main() {
    int x,n;
    printf("Entre x e n: ");
    scanf("%d %d",&x,&n);
    printf("%d",elevado(x,n));
    return 0;
}

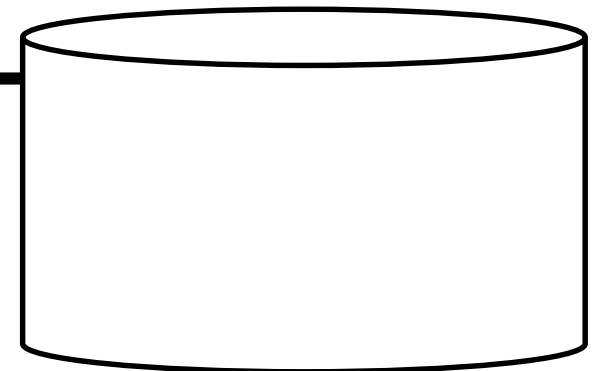
int elevado(int x, int n) {
    int i, resultado;
    resultado = 1;
    for(i = 0; i < n; i++) {
        resultado = x * resultado;
    }
    return resultado;
}
```

Entre x e n: 2 3 <ENTER>
8

Memória

x	2	n	3

HD

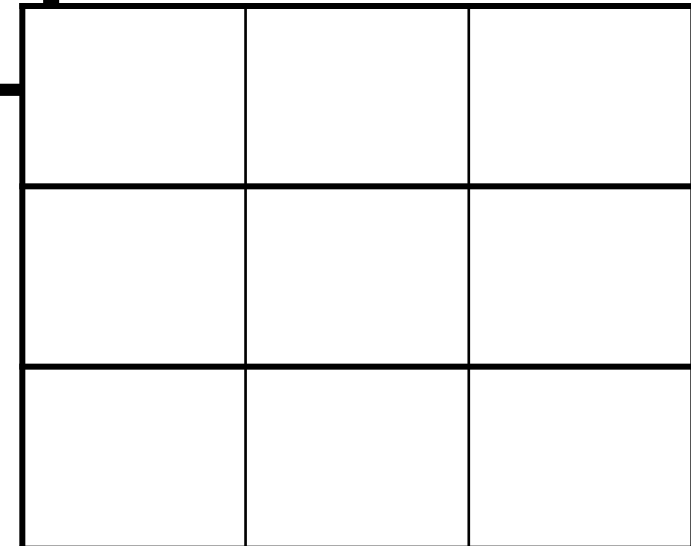


As variáveis de elevado() são apagadas e o equivalente a printf("%d",8) é executado.

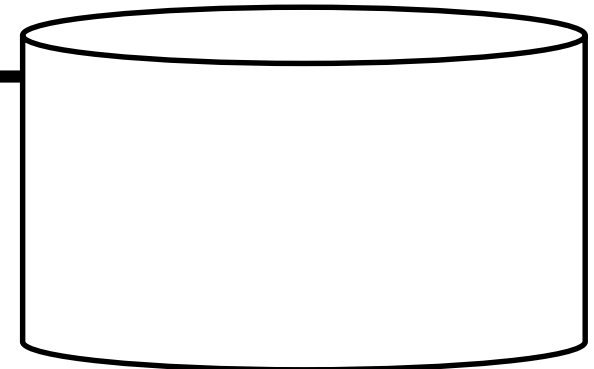
Execução

```
#include <stdio.h>
int elevado(int x, int n);
int main() {
    int x,n;
    printf("Entre x e n: ");
    scanf("%d %d",&x,&n);
    printf("%d",elevado(x,n));
    return 0;
}
int elevado(int x, int n) {
    int i, resultado;
    resultado = 1;
    for(i = 0; i < n; i++) {
        resultado = x * resultado;
    }
    return resultado;
}
```

Memória



HD



Escopo de Variáveis

```
#include <stdio.h>

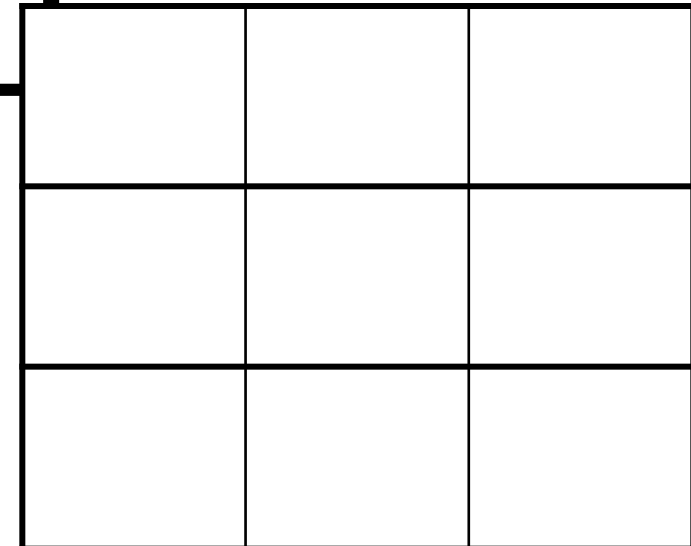
int soma(int x, int y, int z, int w) {
    int k = x + y + z + w;
    return k;
}

int main() {
    int s;
    s = soma(1,2,3,4);
    printf("%d\n",s);
    s = soma(4,3,2,1);
    printf("%d\n",s);
    s = soma(1,1,1,1);
    printf("%d\n",s);
    return 0;
}
```

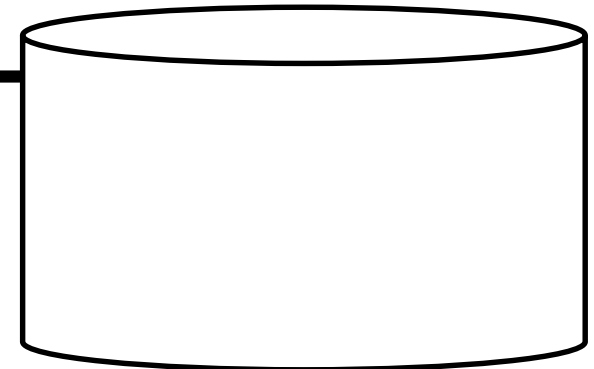
Execução

```
#include <stdio.h>
int soma(int x, int y, int z, int w) {
    int k = x + y + z + w;
    return k;
}
int main() {
    int s;
    s = soma(1,2,3,4);
    printf("%d\n",s);
    s = soma(4,3,2,1);
    printf("%d\n",s);
    s = soma(1,1,1,1);
    printf("%d\n",s);
    return 0;
}
```

Memória

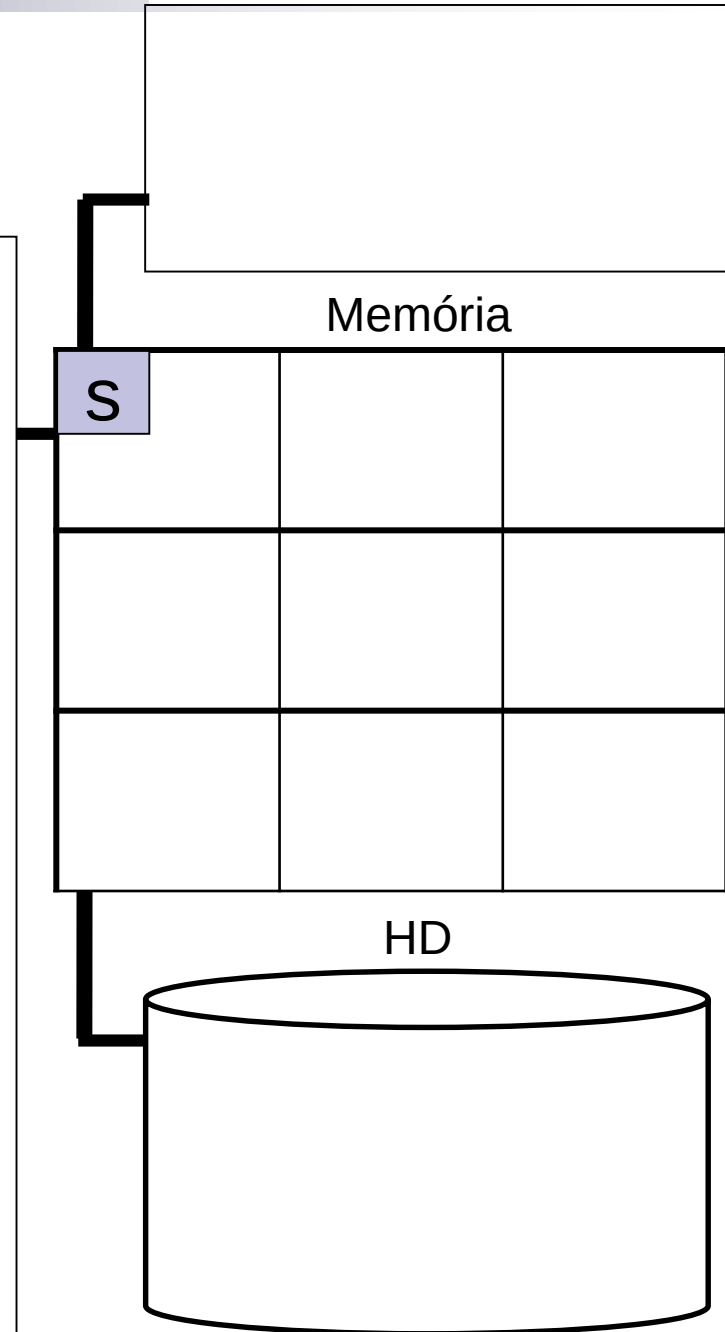


HD



Execução

```
#include <stdio.h>
int soma(int x, int y, int z, int w) {
    int k = x + y + z + w;
    return k;
}
int main() {
    → int s;
    s = soma(1,2,3,4);
    printf("%d\n",s);
    s = soma(4,3,2,1);
    printf("%d\n",s);
    s = soma(1,1,1,1);
    printf("%d\n",s);
    return 0;
}
```



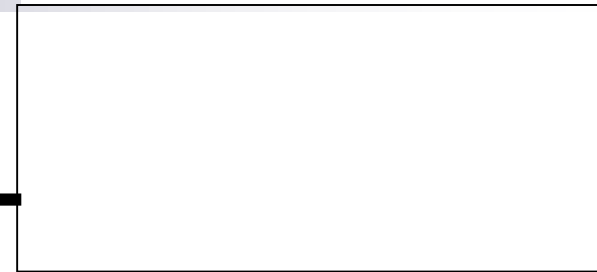
Execução

```
#include <stdio.h>

int soma(int x, int y, int z, int w) {
    int k = x + y + z + w;
    return k;
}

int main() {
    int s;
    s = soma(1, 2, 3, 4);
    printf("%d\n", s);
    s = soma(4, 3, 2, 1);
    printf("%d\n", s);
    s = soma(1, 1, 1, 1);
    printf("%d\n", s);
    return 0;
}
```

Antes de fazer a atribuição, o computador chama soma(1,2,3,4) para trabalhar.



Memória

S

HD

Execução

```
#include <stdio.h>
```

```
int soma(int x, int y, int z, int w) {  
    int k = x + y + z + w;  
    return k;  
}
```

```
int main() {
```

```
    int s;
```

```
    s = soma(1,2,3,4);
```

```
    printf("%d\n",s);
```

```
    s = soma(4,3,2,1);
```

```
    printf("%d\n",s);
```

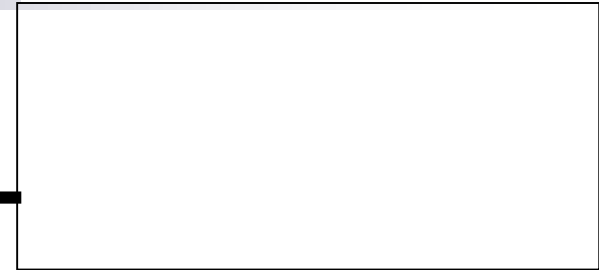
```
    s = soma(1,1,1,1);
```

```
    printf("%d\n",s);
```

```
    return 0;
```

```
}
```

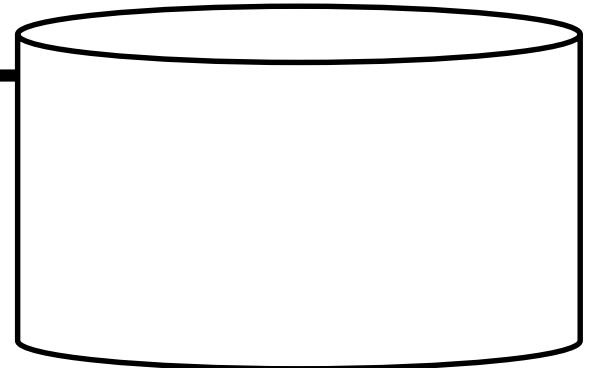
O main() fica parado aqui, aguardando a função soma() retornar.



Memória

S		

HD



Execução

```
#include <stdio.h>
```

```
int soma(int x, int y, int z, int w) {  
    int k = x + y + z + w;  
    return k;  
}
```

```
int main() {  
    int s;
```

```
s = soma(1,2,3,4);  
printf("%d\n",s);  
s = soma(4,3,2,1);  
printf("%d\n",s);  
s = soma(1,1,1,1);  
printf("%d\n",s);  
return 0;
```

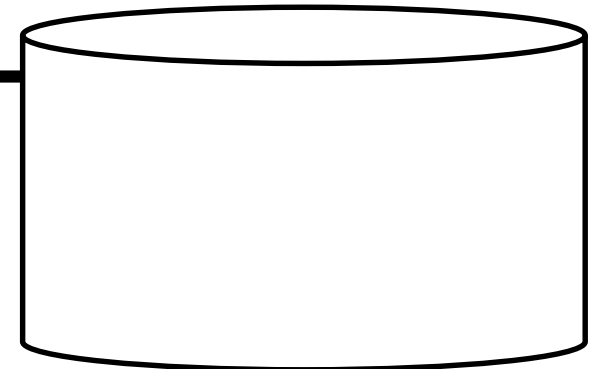
```
}
```

As variáveis são criadas.

Memória

S	X _{soma()}	Y _{soma()}
	1	2
Z _{soma()}	W _{soma()}	
3	4	

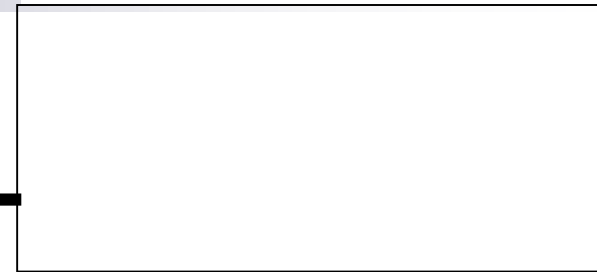
HD



Execução

```
#include <stdio.h>
int soma(int x, int y, int z, int w) {
    int k = x + y + z + w;
    return k;
}
int main() {
    int s;
    s = soma(1,2,3,4);
    printf("%d\n",s);
    s = soma(4,3,2,1);
    printf("%d\n",s);
    s = soma(1,1,1,1);
    printf("%d\n",s);
    return 0;
}
```

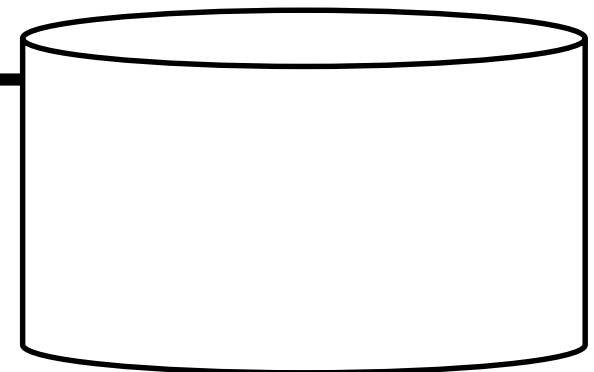
A variável k é criada.



Memória

S	X _{soma()}	Y _{soma()}
	1	2
Z _{soma()}	W _{soma()}	k _{soma()}
3	4	

HD



Execução

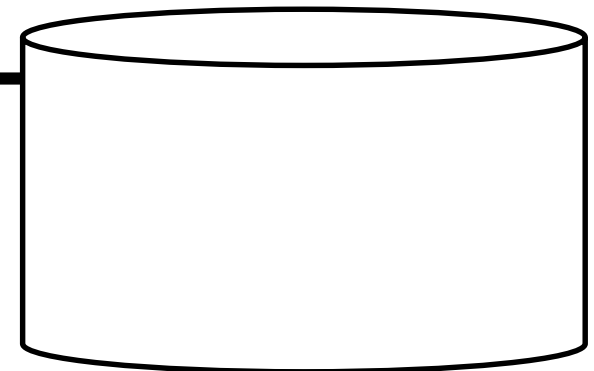
```
#include <stdio.h>
int soma(int x, int y, int z, int w) {
    int k = x + y + z + w;
    return k;
}
int main() {
    int s;
    s = soma(1,2,3,4);
    printf("%d\n",s);
    s = soma(4,3,2,1);
    printf("%d\n",s);
    s = soma(1,1,1,1);
    printf("%d\n",s);
    return 0;
}
```

$$\begin{aligned}
 &X_{\text{soma}()} + Y_{\text{soma}()} + Z_{\text{soma}()} + W_{\text{soma}()} \\
 &= 1 + 2 + 3 + 4 \\
 &= 10
 \end{aligned}$$

Memória

S	X _{soma()}	Y _{soma()}
	1	2
Z _{soma()}	W _{soma()}	k _{soma()}
3	4	

HD



Execução

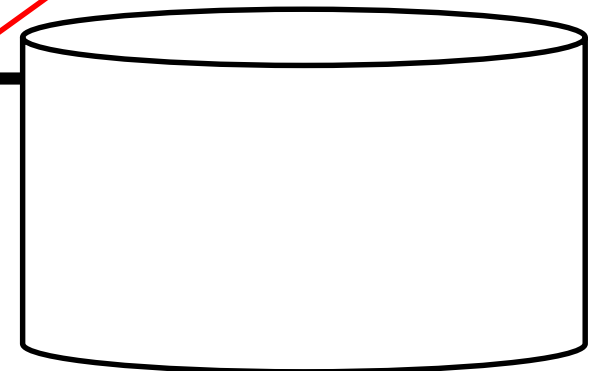
```
#include <stdio.h>
int soma(int x, int y, int z, int w) {
    int k = x + y + z + w;
    return k;
}
int main() {
    int s;
    s = soma(1,2,3,4);
    printf("%d\n",s);
    s = soma(4,3,2,1);
    printf("%d\n",s);
    s = soma(1,1,1,1);
    printf("%d\n",s);
    return 0;
}
```

$$\begin{aligned}
 &X_{\text{soma}()} + Y_{\text{soma}()} + Z_{\text{soma}()} + W_{\text{soma}()} \\
 &= 1 + 2 + 3 + 4 \\
 &= 10
 \end{aligned}$$

Memória

S	X _{soma()}	Y _{soma()}
	1	2
Z _{soma()}	W _{soma()}	k _{soma()}
3	4	10

HD



Execução

```
#include <stdio.h>
int soma(int x, int y, int z, int w) {
    int k = x + y + z + w;
    return k;
}
```

```
int main() {
    int s;
    s = soma(4);
    printf("%d\n", s);
    s = soma(4, 3, 2, 1);
    printf("%d\n", s);
    s = soma(1, 1, 1, 1);
    printf("%d\n", s);
    return 0;
}
```

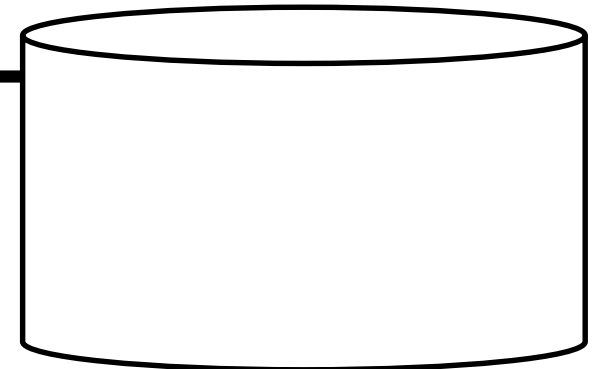
10

O valor de $k_{soma()}$ é
retornado ao `main()` no
lugar da chamada de
`soma()`.

Memória

S	X _{soma()} Y _{soma()}	
	1	2
Z _{soma()}	W _{soma()}	k _{soma()}
3	4	10

HD

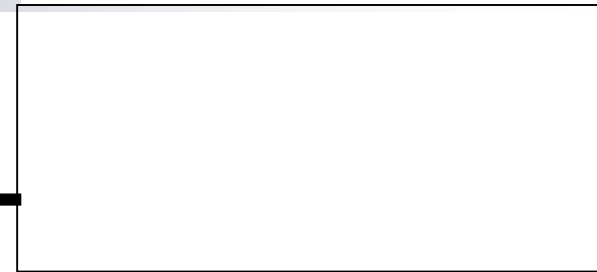


Execução

```
#include <stdio.h>
int soma(int x, int y, int z, int w) {
    int k = x + y + z + w;
    return k;
}
int main() {
    int s;
    s = soma(4, 3, 2, 1);
    printf("%d\n", s);
    s = soma(4, 3, 2, 1);
    printf("%d\n", s);
    s = soma(1, 1, 1, 1);
    printf("%d\n", s);
    return 0;
}
```

10

As variáveis de soma() são
apagadas e o computador
faz o equivalente a
 $s = 10$;



Memória

S

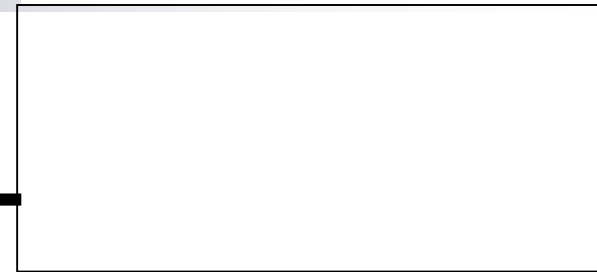
HD

Execução

```
#include <stdio.h>
int soma(int x, int y, int z, int w) {
    int k = x + y + z + w;
    return k;
}
int main() {
    int s;
    s = soma(4, 3, 2, 1);
    printf("%d\n", s);
    s = soma(4, 3, 2, 1);
    printf("%d\n", s);
    s = soma(1, 1, 1, 1);
    printf("%d\n", s);
    return 0;
}
```

10

As variáveis de soma() são apagadas e o computador faz o equivalente a $s = 10$;



Memória

S

10

HD

Execução

```
#include <stdio.h>
int soma(int x, int y, int z, int w) {
    int k = x + y + z + w;
    return k;
}
int main() {
    int s;
    s = soma(1,2,3,4);
    printf("%d\n",s);
    s = soma(4,3,2,1);
    printf("%d\n",s);
    s = soma(1,1,1,1);
    printf("%d\n",s);
    return 0;
}
```

10

Memória

S

10

HD

Execução

```
#include <stdio.h>

int soma(int x, int y, int z, int w) {
    int k = x + y + z + w;
    return k;
}

int main() {
    int s;
    s = soma(1,2,3,4);
    printf("%d\n",s);
    s = soma(4,3,2,1);
    printf("%d\n",s);
    s = soma(1,1,1,1);
    printf("%d\n",s);
    return 0;
}
```

Antes de fazer a atribuição, o computador chama soma(4,3,2,1) para trabalhar.

10

Memória

S

10

HD

Execução

```
#include <stdio.h>

int soma(int x, int y, int z, int w) {
    int k = x + y + z + w;
    return k;
}

int main() {
    int s;
    s = soma(1,2,3,4);
    printf("%d\n",s);
    s = soma(4,3,2,1);
    printf("%d\n",s);
    s = soma(1,1,1,1);
    printf("%d\n",s);
    return 0;
}
```

O main() fica parado aqui, aguardando a função soma() retornar.

10

Memória

S

10

HD

Execução

```
#include <stdio.h>
```

```
int soma(int x, int y, int z, int w) {
    int k = x + y + z + w;
    return k;
}
```

```
int main() {
    int s;
    s = soma(1,2,3,4);
    printf("%d\n",s);
    s = soma(4,3,2,1);
    printf("%d\n",s);
    s = soma(1,1,1,1);
    printf("%d\n",s);
    return 0;
}
```

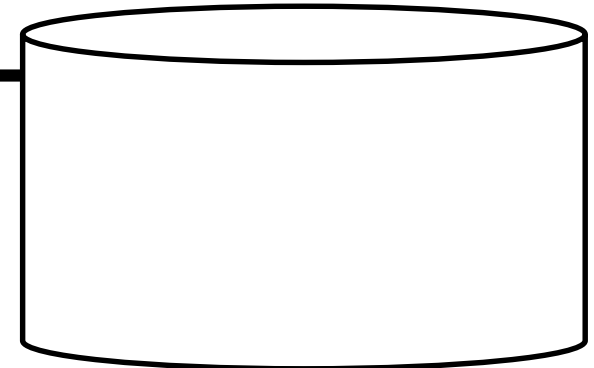
As variáveis são criadas.

10

Memória

S	X _{soma()}	Y _{soma()}
10	4	3
Z _{soma()}	W _{soma()}	
2	1	

HD



Execução

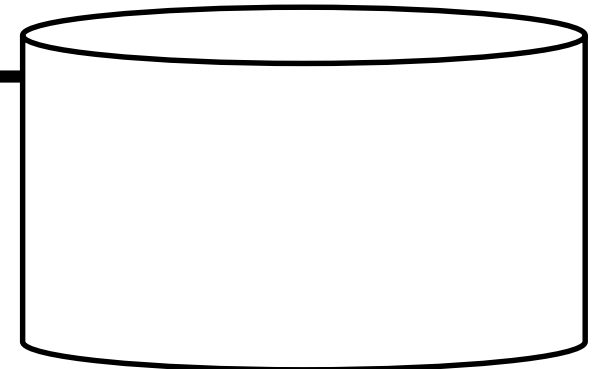
```
#include <stdio.h>
int soma(int x, int y, int z, int w) {
    int k = x + y + z + w;
    return k;
}
int main() {
    int s;
    s = soma(1,2,3,4);
    printf("%d\n",s);
    s = soma(4,3,2,1);
    printf("%d\n",s);
    s = soma(1,1,1,1);
    printf("%d\n",s);
    return 0;
}
```

10

Memória

S	X _{soma()}	Y _{soma()}
10	4	3
Z _{soma()}	W _{soma()}	k _{soma()}
2	1	

HD



A variável k é criada.

Execução

```
#include <stdio.h>
int soma(int x, int y, int z, int w) {
    int k = x + y + z + w;
    return k;
}
int main() {
    int s;
    s = soma(1,2,3,4);
    printf("%d\n",s);
    s = soma(4,3,2,1);
    printf("%d\n",s);
    s = soma(1,1,1,1);
    printf("%d\n",s);
    return 0;
}
```

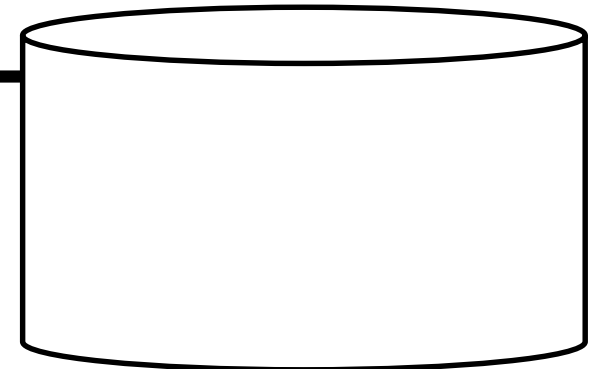
$$\begin{aligned}
 & x_{\text{soma}()} + y_{\text{soma}()} + z_{\text{soma}()} + w_{\text{soma}()} \\
 &= 4 + 3 + 2 + 1 \\
 &= 10
 \end{aligned}$$

10

Memória

S	X _{soma()}	Y _{soma()}
10	4	3
Z _{soma()}	W _{soma()}	k _{soma()}
2	1	

HD



Execução

```
#include <stdio.h>
int soma(int x, int y, int z, int w) {
    int k = x + y + z + w;
    return k;
}
int main() {
    int s;
    s = soma(1,2,3,4);
    printf("%d\n",s);
    s = soma(4,3,2,1);
    printf("%d\n",s);
    s = soma(1,1,1,1);
    printf("%d\n",s);
    return 0;
}
```

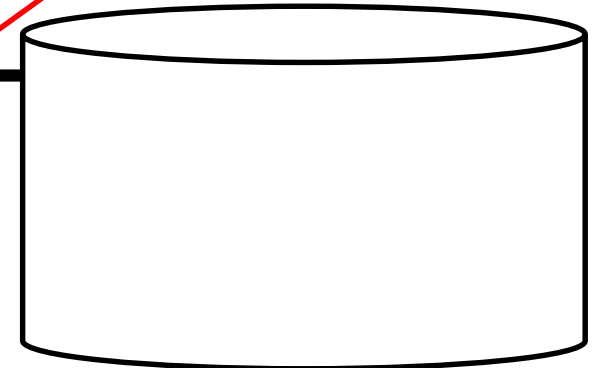
$$\begin{aligned}
 &X_{\text{soma()}} + Y_{\text{soma()}} + Z_{\text{soma()}} + W_{\text{soma()}} \\
 &= 4 + 3 + 2 + 1 \\
 &= 10
 \end{aligned}$$

10

Memória

S	X _{soma()}	Y _{soma()}
10	4	3
Z _{soma()}	W _{soma()}	k _{soma()}
2	1	10

HD



Execução

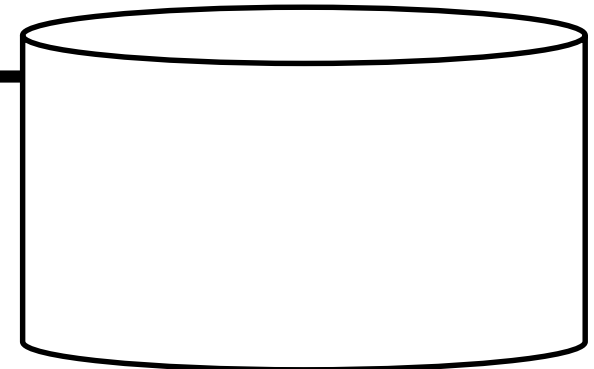
```
#include <stdio.h>
int soma(int x, int y, int z, int w) {
    int k = x + y + z + w;
    return k;
}
int main() {
    int s;
    s = soma(1,2,3,4);
    printf("%d\n",s);
    s = soma(1,1,1,1);
    printf("%d\n",s);
    s = soma(1,1,1,1);
    printf("%d\n",s);
    return 0;
}
```

10

Memória

S	X _{soma()}	Y _{soma()}
10	4	3
Z _{soma()}	W _{soma()}	k _{soma()}
2	1	10

HD



O valor de $k_{soma()}$ é
retornado ao `main()` no
lugar da chamada de
`soma()`.

Execução

```
#include <stdio.h>
int soma(int x, int y, int z, int w) {
    int k = x + y + z + w;
    return k;
}
int main() {
    int s;
    s = soma(1,2,3,4);
    printf("%d\n",s); 10
    s = soma(1,2,3,4);
    printf("%d\n",s);
    s = soma(1,1,1,1);
    printf("%d\n",s);
    return 0;
}
```

*As variáveis de soma() são apagadas e o computador faz o equivalente a
s = 10;*

10

Memória

S

10

HD

Execução

```
#include <stdio.h>

int soma(int x, int y, int z, int w) {
    int k = x + y + z + w;
    return k;
}

int main() {
    int s;
    s = soma(1,2,3,4);
    printf("%d\n",s);
    s = soma(4,3,2,1);
    printf("%d\n",s);
    s = soma(1,1,1,1);
    printf("%d\n",s);
    return 0;
}
```

10
10

Memória

S

10

HD

Execução

```
#include <stdio.h>

int soma(int x, int y, int z, int w) {
    int k = x + y + z + w;
    return k;
}

int main() {
    int s;
    s = soma(1,2,3,4);
    printf("%d\n",s);
    s = soma(4,3,2,1);
    printf("%d\n",s);
    s = soma(1,1,1,1);
    printf("%d\n",s);
    return 0;
}
```

Antes de fazer a atribuição, o computador chama soma(1,1,1,1) para trabalhar.

10
10

Memória

S

10

HD

Execução

```
#include <stdio.h>
```

```
int soma(int x, int y, int z, int w) {  
    int k = x + y + z + w;  
    return k;  
}
```

```
int main() {  
    int s;  
    s = soma(1,2,3,4);  
    printf("%d\n",s);  
    s = soma(4,3,2,1);  
    printf("%d\n",s);  
    s = soma(1,1,1,1);  
    printf("%d\n",s);  
    return 0;  
}
```

O main() fica parado aqui, aguardando a função soma() retornar.

10
10

Memória

S

10

HD

Execução

```
#include <stdio.h>
```

```
int soma(int x, int y, int z, int w) {
    int k = x + y + z + w;
    return k;
}
```

```
int main() {
    int s;
    s = soma(1,2,3,4);
    printf("%d\n",s);
    s = soma(4,3,2,1);
    printf("%d\n",s);
    s = soma(1,1,1,1);
    printf("%d\n",s);
    return 0;
}
```

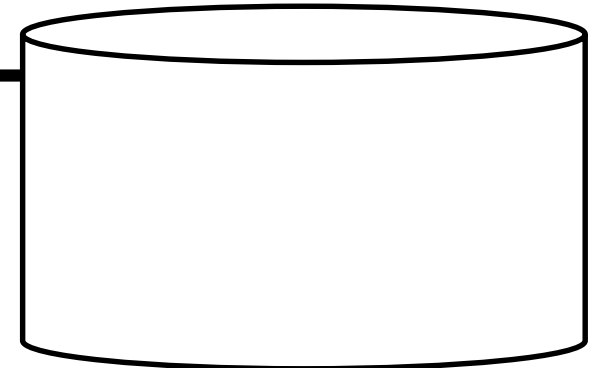
As variáveis são criadas.

10
10

Memória

S	X _{soma()}	Y _{soma()}
10	1	1
Z _{soma()}	W _{soma()}	
1	1	

HD



Execução

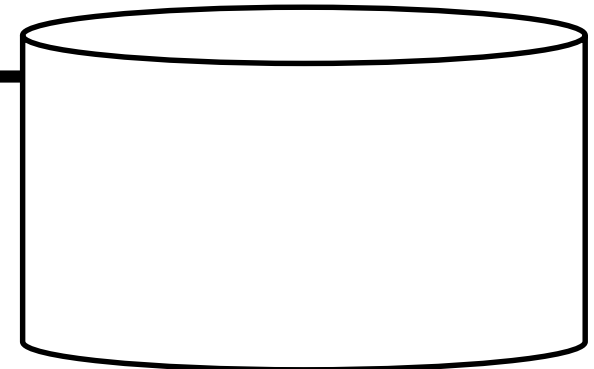
```
#include <stdio.h>
int soma(int x, int y, int z, int w) {
    int k = x + y + z + w;
    return k;
}
int main() {
    int s;
    s = soma(1,2,3,4);
    printf("%d\n",s);
    s = soma(4,3,2,1);
    printf("%d\n",s);
    s = soma(1,1,1,1);
    printf("%d\n",s);
    return 0;
}
```

10
10

Memória

S	X _{soma()}	Y _{soma()}
10	1	1
Z _{soma()}	W _{soma()}	k _{soma()}
1	1	

HD



A variável k é criada.

Execução

```
#include <stdio.h>
int soma(int x, int y, int z, int w) {
    int k = x + y + z + w;
    return k;
}
int main() {
    int s;
    s = soma(1,2,3,4);
    printf("%d\n",s);
    s = soma(4,3,2,1);
    printf("%d\n",s);
    s = soma(1,1,1,1);
    printf("%d\n",s);
    return 0;
}
```

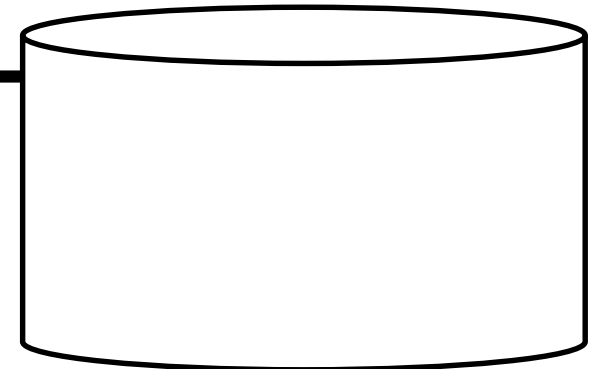
$$\begin{aligned}
 &X_{\text{soma()}} + Y_{\text{soma()}} + Z_{\text{soma()}} + W_{\text{soma()}} \\
 &= 1 + 1 + 1 + 1 \\
 &= 4
 \end{aligned}$$

10
10

Memória

S	X _{soma()}	Y _{soma()}
10	1	1
Z _{soma()}	W _{soma()}	k _{soma()}
1	1	

HD



Execução

```
#include <stdio.h>
int soma(int x, int y, int z, int w) {
    int k = x + y + z + w;
    return k;
}
int main() {
    int s;
    s = soma(1,2,3,4);
    printf("%d\n",s);
    s = soma(4,3,2,1);
    printf("%d\n",s);
    s = soma(1,1,1,1);
    printf("%d\n",s);
    return 0;
}
```

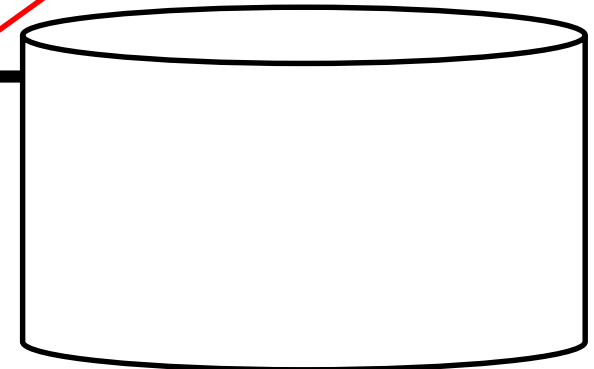
$$\begin{aligned}
 &X_{soma()} + Y_{soma()} + Z_{soma()} + W_{soma()} \\
 &= 1 + 1 + 1 + 1 \\
 &= 4
 \end{aligned}$$

10
10

Memória

S	X _{soma()}	Y _{soma()}
10	1	1
Z _{soma()}	W _{soma()}	k _{soma()}
1	1	4

HD



Execução

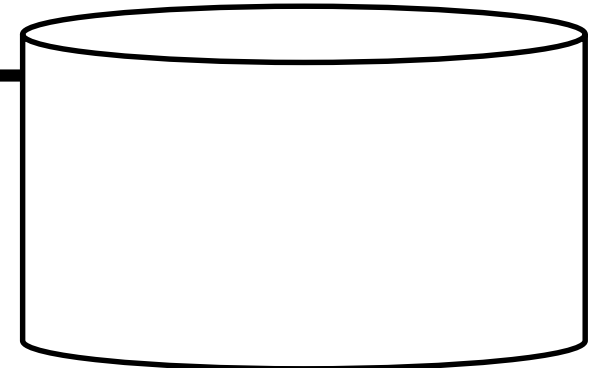
```
#include <stdio.h>
int soma(int x, int y, int z, int w) {
    int k = x + y + z + w;
    return k;
}
int main() {
    int s;
    s = soma(1,2,3,4);
    printf("%d\n",s);
    s = soma(4,3,2,1);
    printf("%d\n",s);
    s = soma(1,2,3,4); 4
    printf("%d\n",s);
    return 0;
}
```

10
10

Memória

S	X _{soma()}	Y _{soma()}
10	1	1
Z _{soma()}	W _{soma()}	k _{soma()}
1	1	4

HD



O valor de $k_{soma()}$ é
retornado ao main() no
lugar da chamada de
soma().

Execução

```
#include <stdio.h>
int soma(int x, int y, int z, int w) {
    int k = x + y + z + w;
    return k;
}
int main() {
    int s;
    s = soma(1,2,3,4);
    printf("%d\n",s);
    s = soma(4,3,2,1);
    printf("%d\n",s);
    s = soma(1,2,3,4); 4
    printf("%d\n",s);
    return 0;
}
```

As variáveis de soma() são apagadas e o computador faz o equivalente a $s = 4$;

10
10

Memória

S

4

HD

Execução

```
#include <stdio.h>

int soma(int x, int y, int z, int w) {
    int k = x + y + z + w;
    return k;
}

int main() {
    int s;
    s = soma(1,2,3,4);
    printf("%d\n",s);
    s = soma(4,3,2,1);
    printf("%d\n",s);
    s = soma(1,1,1,1);
    printf("%d\n",s);
    return 0;
}
```

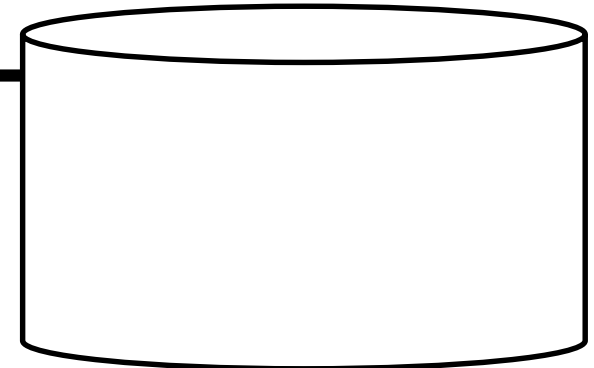
10
10
4

Memória

S

4

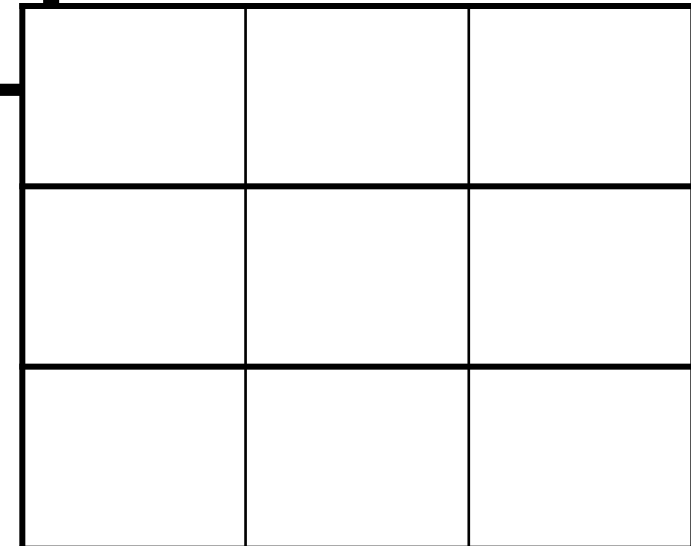
HD



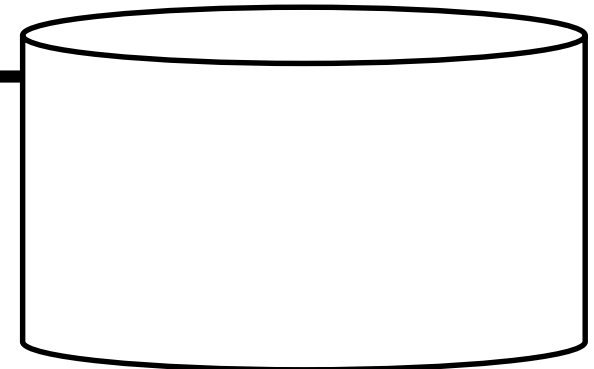
Execução

```
#include <stdio.h>
int soma(int x, int y, int z, int w) {
    int k = x + y + z + w;
    return k;
}
int main() {
    int s;
    s = soma(1,2,3,4);
    printf("%d\n",s);
    s = soma(4,3,2,1);
    printf("%d\n",s);
    s = soma(1,1,1,1);
    printf("%d\n",s);
    return 0;
}
```

Memória



HD





Variáveis globais

- Uso não é aconselhável
 - Dificulta o entendimento do programa
 - Qualquer função pode modificar o valor da variável global
 - Podem ser cobertas por outras variáveis
- Dessa forma, evite o uso destas variáveis

Escopo de Variáveis

```
#include <stdio.h>
```

```
int j = 0;
```

```
void soma(int x) {
```

```
    j = j + x;
```

```
}
```

```
int main() {
```

```
    int x = 3;
```

```
    soma(x);
```

```
    printf("j = %d \n", j);
```

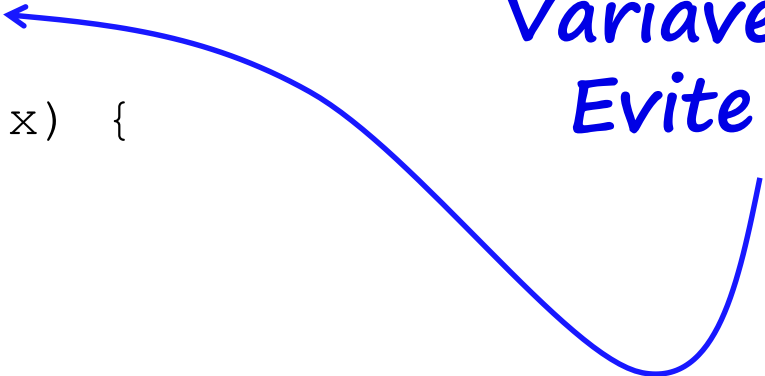
```
    j = 1;
```

```
    printf("j = %d \n", j);
```

```
    return 0;
```

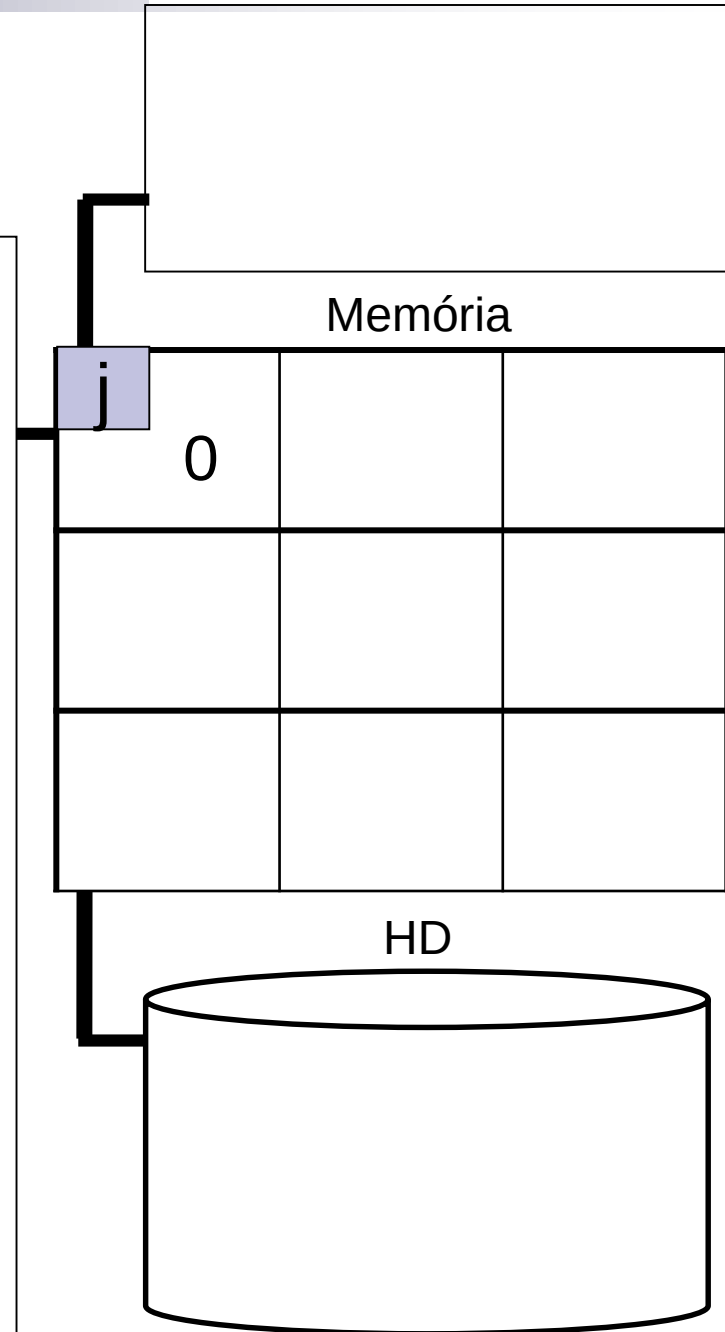
```
}
```

*Variável global.
Evite o uso!*



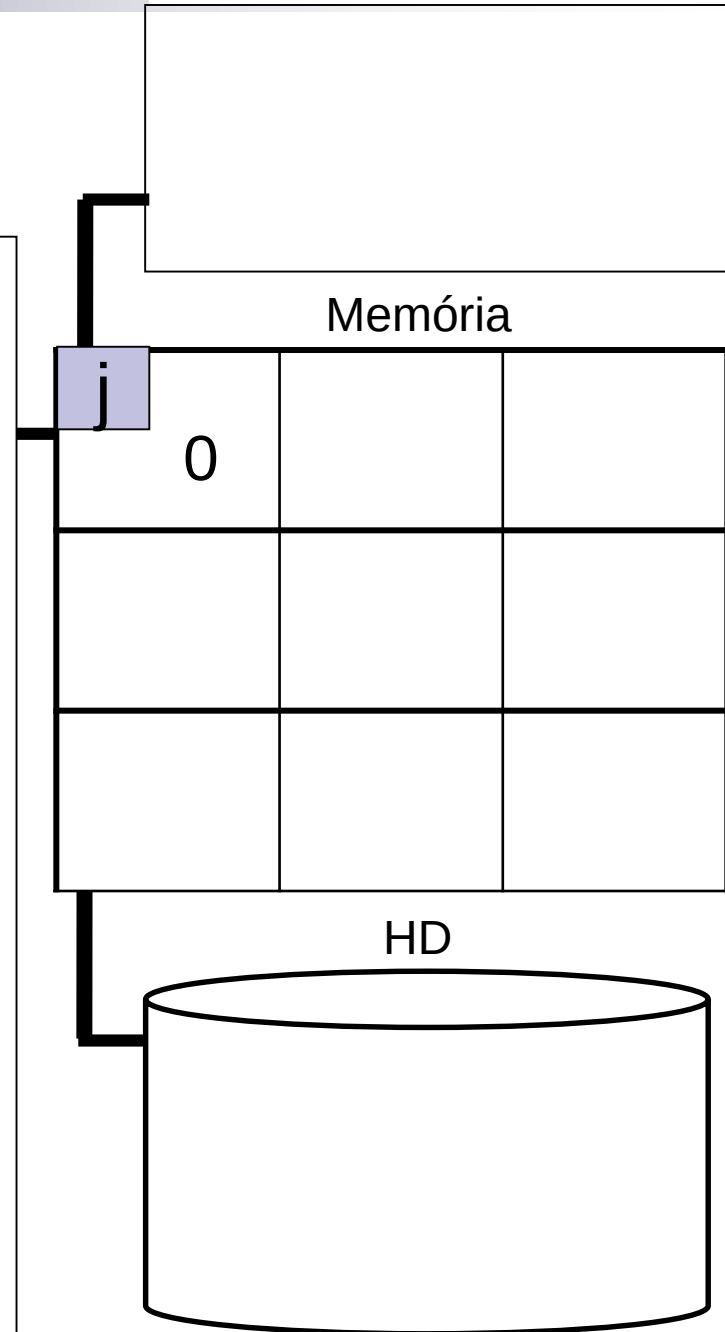
Execução

```
#include <stdio.h>
→ int j = 0;
void soma(int x) {
    j = j + x;
}
int main() {
    int x = 3;
    soma(x);
    printf("j = %d \n", j);
    j = 1;
    printf("j = %d \n", j);
    return 0;
}
```



Execução

```
#include <stdio.h>
int j = 0;
void soma(int x) {
    j = j + x;
}
→ int main() {
    int x = 3;
    soma(x);
    printf("j = %d \n", j);
    j = 1;
    printf("j = %d \n", j);
    return 0;
}
```



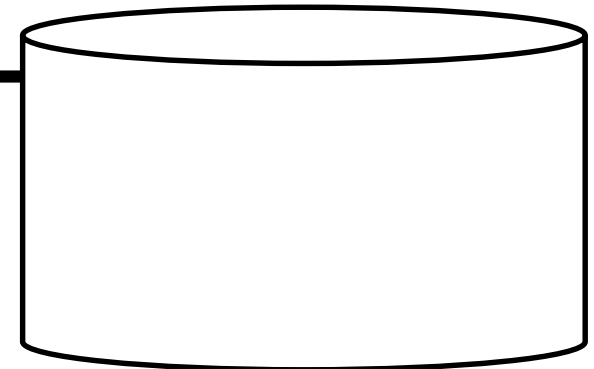
Execução

```
#include <stdio.h>
int j = 0;
void soma(int x) {
    j = j + x;
}
int main() {
    int x = 3;
    soma(x);
    printf("j = %d \n", j);
    j = 1;
    printf("j = %d \n", j);
    return 0;
}
```

Memória

j	0	x	3	

HD



Execução

```
#include <stdio.h>
```

```
int j = 0;
```

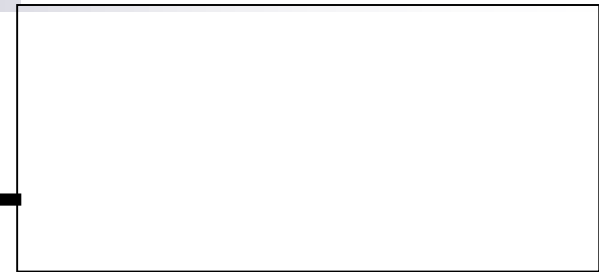
```
→ void soma(int x) {  
    j = j + x;  
}
```

```
int main() {
```

```
    int x = 3;
```

```
→ soma(x);  
    printf("j = %d \n", j);  
    j = 1;  
    printf("j = %d \n", j);  
    return 0;  
}
```

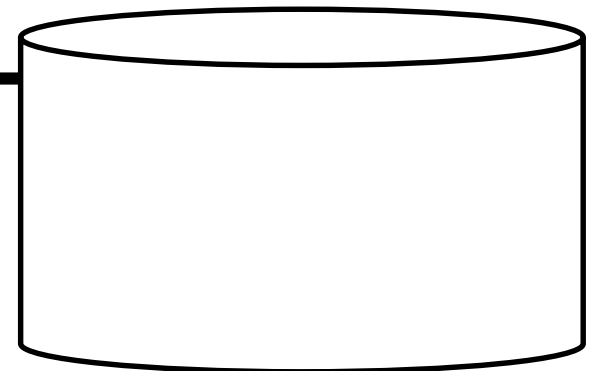
O main() fica parado aqui, aguardando a função soma() retornar.



Memória

j	x	
0	3	

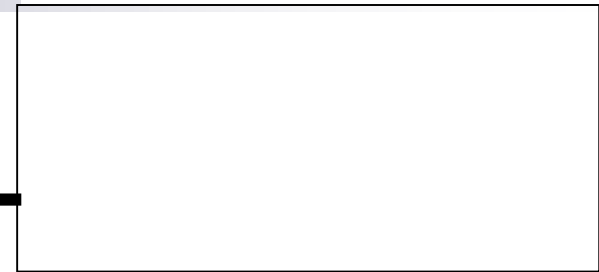
HD



Execução

```
#include <stdio.h>
int j = 0;
void soma(int x) {
    j = j + x;
}
int main() {
    int x = 3;
    soma(x);
    printf("j = %d \n", j);
    j = 1;
    printf("j = %d \n", j);
    return 0;
}
```

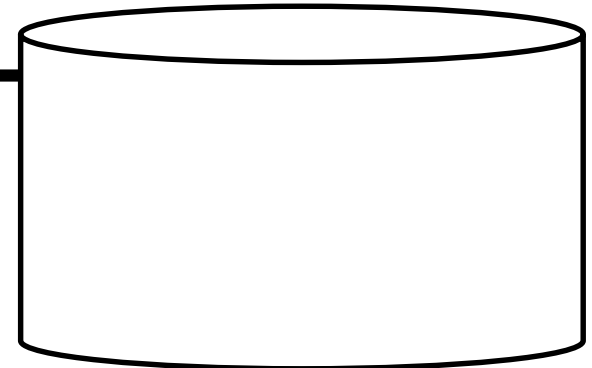
A variável x é criada.



Memória

j	x	X _{soma()}
0	3	3

HD



Execução

```
#include <stdio.h>
```

```
int j = 0;
```

```
void soma(int x) {
```

```
    j = j + x;
```

```
}
```

```
int main() {
```

```
    int x = 3;
```

```
    soma(x);
```

```
    printf("j = %d \n", j);
```

```
    j = 1;
```

```
    printf("j = %d \n", j);
```

```
    return 0;
```

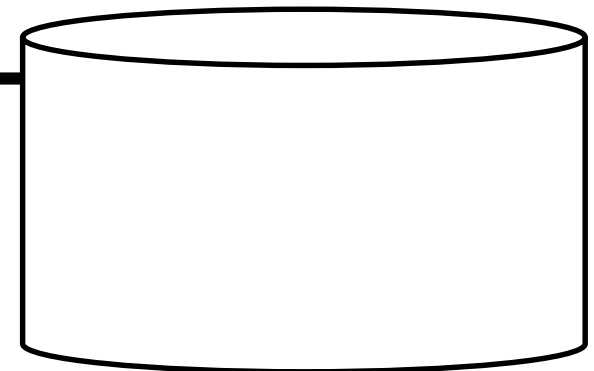
```
}
```

$$\begin{aligned}
 j + x_{\text{soma}()} \\
 &= 0 + 3 \\
 &= 3
 \end{aligned}$$

Memória

j	x	x _{soma()}
0	3	3

HD



Execução

```
#include <stdio.h>
```

```
int j = 0;
```

```
void soma(int x) {
```

→ `j = j + x;`

```
}
```

```
int main() {
```

```
    int x = 3;
```

→ `soma(x);`

```
    printf("j = %d \n", j);
```

```
    j = 1;
```

```
    printf("j = %d \n", j);
```

```
    return 0;
```

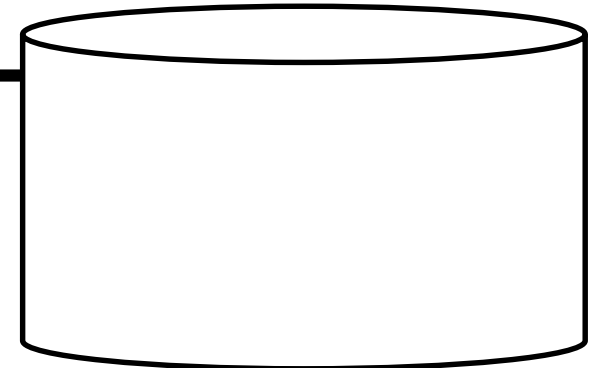
```
}
```

$$j + x_{soma()} = 0 + 3 = 3$$

Memória

j	x	x _{soma()}
3	3	3

HD



Execução

```
#include <stdio.h>
```

```
int j = 0;
```

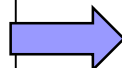
```
void soma(int x) {
```

```
    j = j + x;
```

```
}
```

```
int main() {
```

```
    int x = 3;
```



```
soma(x);
```

```
    printf("j = %d \n", j);
```

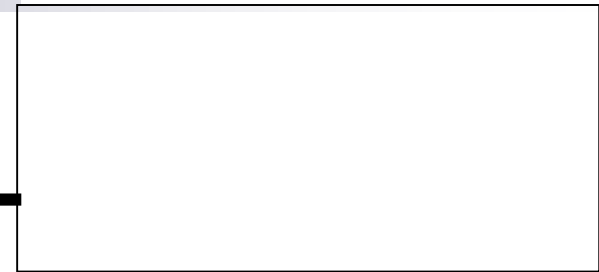
```
    j = 1;
```

```
    printf("j = %d \n", j);
```

```
    return 0;
```

```
}
```

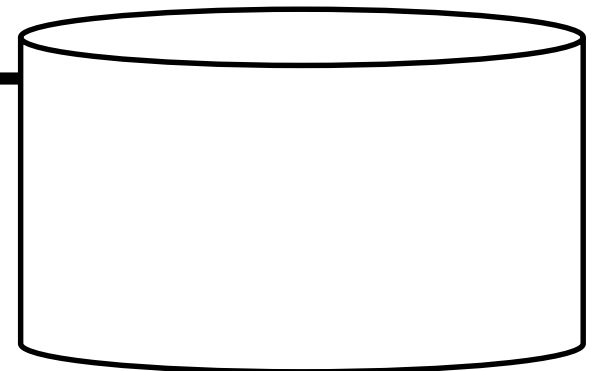
O computador retorna para o main(). Como esta versão de soma() retorna nada (void), o computador prossegue.



Memória

j	X	
3	3	

HD



Execução

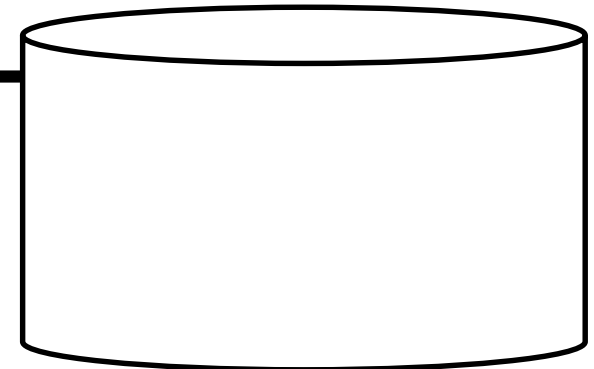
```
#include <stdio.h>
int j = 0;
void soma(int x) {
    j = j + x;
}
int main() {
    int x = 3;
    soma(x);
    printf("j = %d \n", j);
    j = 1;
    printf("j = %d \n", j);
    return 0;
}
```

j = 3

Memória

j	3	X	3	

HD



Execução

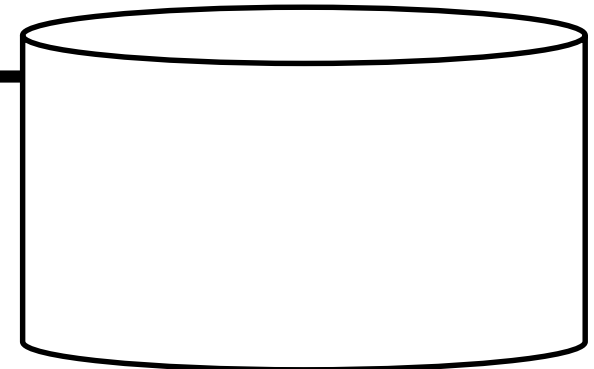
```
#include <stdio.h>
int j = 0;
void soma(int x) {
    j = j + x;
}
int main() {
    int x = 3;
    soma(x);
    printf("j = %d \n", j);
    j = 1;
    printf("j = %d \n", j);
    return 0;
}
```

j = 3

Memória

j	x	
1	3	

HD



Execução

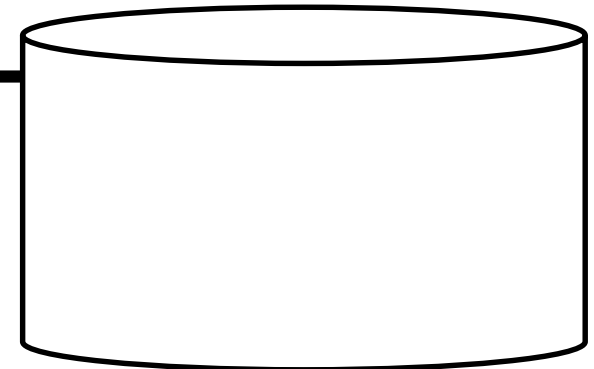
```
#include <stdio.h>
int j = 0;
void soma(int x) {
    j = j + x;
}
int main() {
    int x = 3;
    soma(x);
    printf("j = %d \n", j);
    j = 1;
    printf("j = %d \n", j);
    return 0;
}
```

j = 3
j = 1

Memória

j	1	X	3

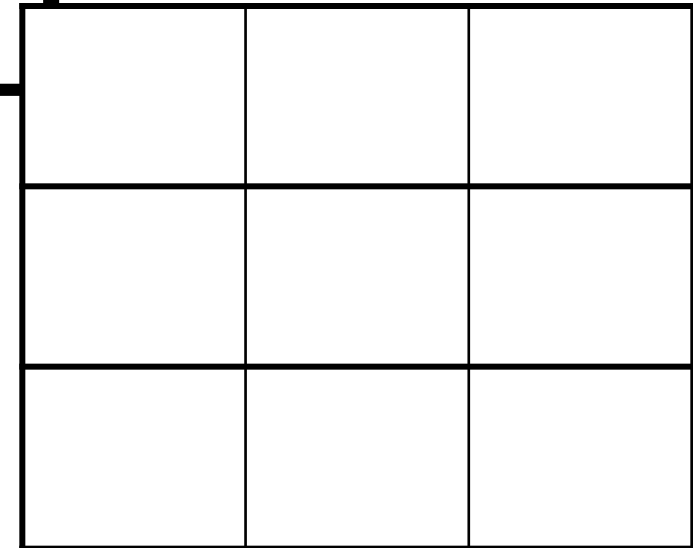
HD



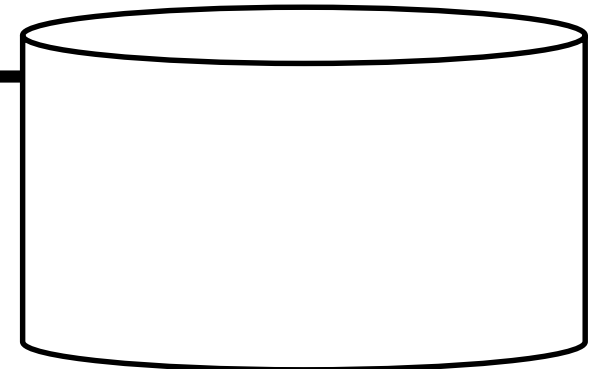
Execução

```
#include <stdio.h>
int j = 0;
void soma(int x) {
    j = j + x;
}
int main() {
    int x = 3;
    soma(x);
    printf("j = %d \n", j);
    j = 1;
    printf("j = %d \n", j);
    return 0;
}
```

Memória



HD



Exercício

- Faça a função `meu_scan()` que
 - Recebe dois inteiros `x` e `y` como parâmetros
 - Retorna um inteiro que foi lido do teclado e que encontra-se dentro do intervalo `[x, y]`
 - Caso o valor digitado esteja fora do intervalo, pede-se novamente até que o valor esteja correto
 - Assuma que os valores dos parâmetros sempre são com $x < y$
- Na função `main()`, chame `meu_scan()` para ler um inteiro maior ou igual a zero e menor ou igual a 1.000 e depois imprima este número

Exercício

- Faça a função `inverte_num()` que
 - Recebe um inteiro de 3 dígitos como parâmetro
 - Retorna o inverso do número (exemplo: o inverso de 765 é 567)
- Na função `main()`, leia um inteiro de 3 dígitos, chame `inverte_num()` e depois imprima o inverso do número digitado

Exercício

- Faça a função `maior2()` que
 - Recebe 2 números e retorna o maior valor
- Faça a função `maior3()` que
 - Recebe 3 números e retorna o maior valor
 - Use a função `maior2()`
- Na função `main()`, leia 3 números do teclado, chame `maior3()` e imprima o maior número digitado